

랭그래프로 목차를 작성하는 멀티에이전트 만들기

크고 복잡한 일을 할 때는 여러 전문가가 작업 결과를 서로 리뷰하고 회의를 하며 진행합니다. 인공지능도 마찬가지입니다. 각기 다른 능력을 갖춘 인공지능 에이전트들끼리 서로 회의하도록 만들면 복잡한 작업도 쉽게 해낼 수 있습니다. 이번 장에서는 책의 목차를 작성하는 멀티에이전트를 만드는 방법을 다루겠습니다.



- 14-1 사용자와 함께 목차를 작성하는 에이전트
- 14-2 조장 역할을 하는 슈퍼바이저 에이전트
- 14-3 웹 검색과 RAG를 활용하는 벡터 검색 에이전트
- 14-4 부족한 정보를 검색하는 웹 검색 에이전트



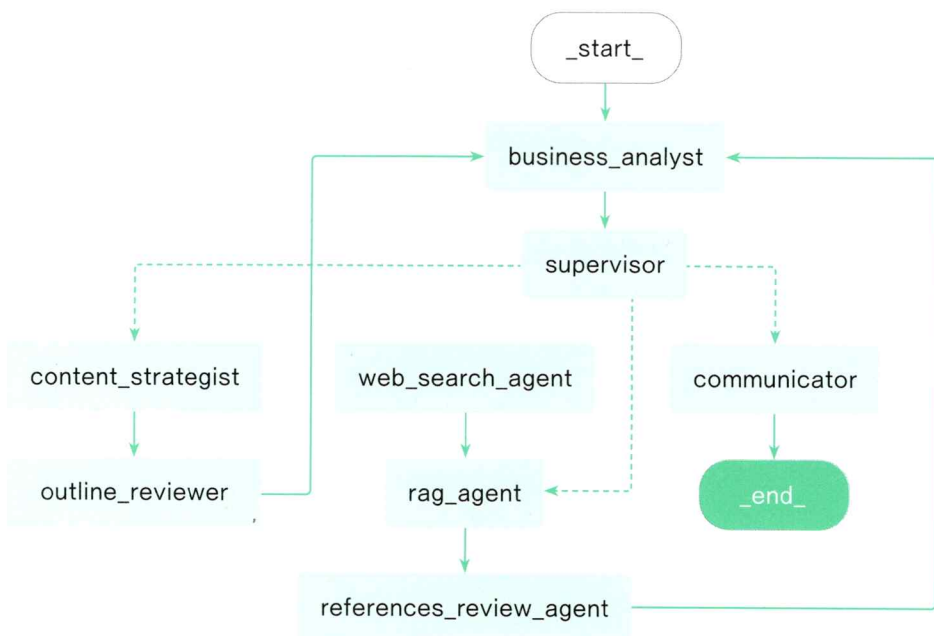
14-1 사용자와 함께 목차를 작성하는 에이전트

‘천 리 길도 한 걸음부터’라는 말이 있습니다. 먼저 이 장에서 만들 멀티에이전트에 대해 알아보고 사용자의 질문에 대답하는 챗봇을 만든 다음, 사용자와 함께 목차를 만드는 챗봇을 완성해 보겠습니다.

이 장에서 만드는 멀티에이전트

이어지는 실습에서는 작은 단위의 인공지능 프로그램들이 서로 협업하여 작업을 수행하는 멀티에이전트로 발전하는 과정을 살펴보겠습니다. 이를 통해 멀티에이전트가 단순한 요구 사항부터 매우 복잡한 업무까지 어떻게 수행해 내는지 알아보겠습니다.

다음 그림은 ‘OOO에 관한 보고서를 써야 하니까 인터넷에서 자료를 조사하고 목차를 작성해 와.’라는 한 문장을 수행하기 위해서 조직된 AI 멀티에이전트의 시스템입니다. 각각의 노드(에이전트)는 고유한 임무를 수행하고 자신의 작업 결과를 다른 AI 에이전트들과 공유하면서 작업을 완수합니다.

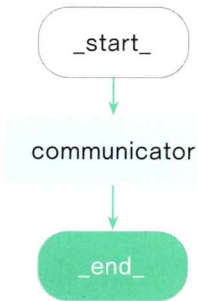


에이전트들이 결과를 공유하며 요청을 수행하는 멀티에이전트의 구조

이 장에서는 AI 에이전트들이 개별 역할을 각각 수행하면서 조화를 이루어 복잡한 업무를 처리하는 과정을 직접 구현해 볼 수 있습니다. 이를 통해 인공지능을 활용한 과업 수행의 기본 원리를 이해하고, 더 나아가 여러분의 프로젝트에 실제로 적용할 아이디어를 얻을 수 있을 것입니다.

사용자와 의사소통하는 커뮤니케이터 에이전트

사용자와 단순한 대화를 할 수 있는 커뮤니케이터 에이전트 communicator를 만들겠습니다. 이어지는 실습에서 기능을 계속 추가할 예정입니다. 사용자가 메시지를 입력하면 커뮤니케이터 에이전트가 답변을 생성하고 결과를 내놓는 단순한 구조입니다.



communicator의 그래프 구조

Do it! 실습 커뮤니케이터 에이전트 communicator 만들기

■ 결과 파일: chap14/sec01/book_writer_0.py, utils_0.py

1. 새로운 파이썬 파일인 book_writer.py을 만들고 다음과 같이 코드를 작성합니다. 대부분은 12-1절에서 챗봇을 만들 때 작성한 코드를 가져온 것입니다. 새로 등장한 코드 위주로 살펴보겠습니다.

현재 폴더 경로 찾기

book_writer.py

```
from langgraph.graph import StateGraph, START, END
from langchain_openai import ChatOpenAI
from langchain_core.messages import AnyMessage, SystemMessage, HumanMessage
from langchain_core.prompts import PromptTemplate
from typing_extensions import TypedDict
from typing import List

from utils import save_state ①
```

```

from datetime import datetime
import os

# 현재 폴더 경로 찾기
# 랭그래프 이미지로 저장 및 추후 작업 결과 파일 저장 경로로 활용
filename = os.path.basename(__file__)          # 현재 파일명 반환
absolute_path = os.path.abspath(__file__)       # 현재 파일의 절대 경로 반환
current_path = os.path.dirname(absolute_path)   # 현재 .py 파일이 있는 폴더 경로

# 모델 초기화
llm = ChatOpenAI(model="gpt-4o")

```

- ❶ save_state 함수를 가져올 utils 파일을 임포트합니다. 이 파일은 뒤에서 작성합니다.
- ❷ 현재 경로를 찾기 위한 내용을 추가합니다. 현재 랭그래프의 구조를 이미지로 저장하기 위한 경로를 설정하고 나중에 목차를 만들면 그 결과 파일을 저장할 경로를 설정하기 위해 current_path를 사용할 것입니다. 이 current_path에는 파이썬 파일인 book_writer.py가 있는 경로를 담습니다.

2. 상태를 정의하겠습니다. 앞으로 더 확장해 나가겠지만 현재는 대화 기록을 담아 둘 수 있도록 messages라는 변수에 리스트 자료형을 사용합니다. 이 리스트에 들어갈 수 있는 자료형은 AnyMessage 혹은 문자열(str)입니다. 꼭 랭체인 메시지가 아니라 문자열로 들어오더라도 GPT에서 처리할 수 있으므로 AnyMessage뿐만 아니라 str로도 담을 수 있게 했습니다.

상태 정의하기

book_writer.py

```

(... 생략 ...)
# 상태 정의
class State(TypedDict):
    messages: List[AnyMessage | str]

```

랭체인에서 AnyMessage는 여러 종류의 메시지 타입을 하나로 통합하여 표현하기 위해 사용되는 타입 별칭 **type alias**입니다. 즉, AnyMessage는 다음과 같은 다양한 메시지 클래스를 포함하는 유니온 **union** 타입으로 정의되어 있습니다. AnyMessage를 사용하면 다양한 메시지 타입을 모두 받아들일 수 있어서 코드의 유연성과 가독성을 높일 수 있습니다. 예를 들어 채팅

기록이나 메시지 리스트를 처리할 때 AnyMessage 타입으로 선언하면 앞에 언급한 모든 메시지 객체를 하나의 리스트로 다룰 수 있습니다.

타입	설명
AIMessage	AI(모델)가 생성한 응답 메시지
HumanMessage	사용자(인간)가 입력한 메시지
SystemMessage	시스템에서 모델의 행동이나 대화의 맥락을 지정하는 메시지
ToolMessage	도구(tool)의 호출 결과를 나타내는 메시지

3. 사용자와 대화하는 에이전트 **communicator**를 만들겠습니다. 이 AI 에이전트는 목차를 작성하는 AI 팀의 일원으로 기존 대화 내용을 바탕으로 사용자와 상호 작용하며 대화하는 임무를 수행합니다.

communicator 에이전트 만들기

book_writer.py

```
(... 생략 ...)
# 사용자와 대화할 노드(agent): communicator
def communicator(state: State):
    print("\n\n===== COMMUNICATOR =====")

    communicator_system_prompt = PromptTemplate.from_template(
        """
        너는 책을 쓰는 AI 팀의 커뮤니케이터로서,
        AI 팀의 진행 상황을 사용자에게 보고하고, 사용자의 의견을 파악하기 위해 대화를 나눈다.
        messages: {messages}
        """
    )

    system_chain = communicator_system_prompt | llm

    # 상태에서 메시지 가져오기
    messages = state["messages"]

    # 입력값 정의
    inputs = {"messages": messages}
```

```

gathered = None

print('\nAI\t: ', end='')
for chunk in system_chain.stream(inputs):
    print(chunk.content, end='')

    if gathered is None:
        gathered = chunk
    else:
        gathered += chunk

messages.append(gathered)

return {"messages": messages}

```

- ❶ 임무 내용을 `communicator_system_prompt`에 `PromptTemplate`를 이용해 프롬프트로 정의합니다. 이 프롬프트는 시스템 프롬프트 역할을 합니다.
 - ❷ 이 시스템 프롬프트는 `llm`에 체인으로 연결됩니다. 프롬프트가 기존 대화 내용을 계속 업데이트하고 답변할 수 있도록 `state`에서 `messages`를 받을 수 있게 합니다.
 - ❸ `system_chain.stream(inputs)`로 스트림 출력을 하기 위해 빈 변수 `gathered`를 만들고 언어 모델에서 스트림되는 내용을 터미널 창에 출력하면서 `gathered`에 점차 덧붙입니다. 그리고 `gathered`의 값을 `messages` 리스트에 추가하고 `messages`를 딕셔너리 형태로 반환하여 상태(State)를 업데이트합니다.
- ◆ 매번 `messages`에 값을 추가하지 않고 랭그라프나 랭체인에서 제공하는 `MemorySaver`와 같은 기능을 활용할 수도 있습니다. 메모리에 관한 내용은 12-2절을 참고하세요. 이 방식은 취향에 따라 선택하면 됩니다. 이 예제에서는 어느 시점에 메시지가 대화 내용에 추가되는지 코드에서 확실히 보기 위해 대화 내용을 직접 관리하는 방법을 선택했습니다.

4. 이제 `StateGraph`로 그래프를 만들 차례입니다. 이 그래프는 `Start` → `communicator` → `END`의 단순한 구조로 되어 있습니다. 이번에 만든 `communicator` 노드만 `graph_builder`에 추가하고, 이 노드를 `START`와 `END`에 연결한 뒤 그래프를 컴파일합니다.

그래프 만들기

book_writer.py

```

(... 생략 ...)
# 상태 그래프 정의
graph_builder = StateGraph(State)

# Nodes
graph_builder.add_node("communicator", communicator)

# Edges
graph_builder.add_edge(START, "communicator")
graph_builder.add_edge("communicator", END)

```

```
graph = graph_builder.compile()
```

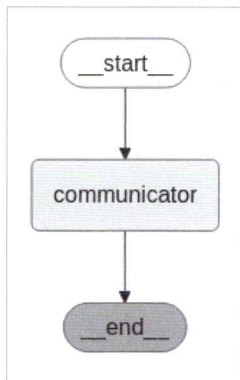
5. 랭그래프를 도식화하는 코드입니다. 12-1절에서 작성한 주피터 노트북 코드(langgraph_simple_chatbot.ipynb)와 거의 동일하지만 도식화된 결과를 PNG 파일로 저장하도록 수정했습니다. 여기에서는 앞서 작성한 `absolute_path`를 사용하며 현재 작업하고 있는 파이썬 파일의 이름과 동일한 PNG 파일을 만듭니다.

그래프 도식화하고 PNG 파일 생성하기

book_writer.py

```
(... 생략 ...)
# 그래프 도식화
graph.get_graph().draw_mermaid_png(output_file_path=absolute_path.replace('.py',
'.png'))
```

이 코드를 실행하면 다음 그림을 얻게 됩니다. 아직 `utils.py` 파일을 만들지 않았으므로 현재 코드에서는 실행되지 않습니다. `utils.py` 파일을 작성한 뒤에 실행해 보세요.



6. 전체 워크플로의 시스템 메시지를 작성합니다. 아직은 AI 에이전트가 `communicator` 하나 뿐이지만 나중에 여러 AI 에이전트가 추가될 것을 고려하여 설계합니다. 그리고 프롬프트에 현재 시각 정보를 포함하도록 하여 향후 책이나 보고서를 만들 때 언어 모델이 만들어진 시점(gpt-4o의 경우 2023년)을 기준으로 판단하는 오류를 방지할 수 있게 합니다.

```
(... 생략 ...)
# 상태 초기화
state = State(
    messages = [
        SystemMessage(
            f"""
            너희 AI들은 사용자의 요구에 맞는 책을 쓰는 작가 팀이다.
            사용자가 사용하는 언어로 대화하라.

            현재 시각은 {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}이다.

            """,
        )
    ],
)
```

7. 터미널 창에서 사용자의 입력을 받고 `graph`를 실행(`invoke`)하는 코드입니다. 사용자가 입력한 값을 `user_input` 변수에 담아 워크플로를 실행합니다. 반복문은 사용자가 입력한 값이 'exit', 'quit', 'q'가 아니면 계속됩니다. 실제로 AI가 생성한 답변은 `communicator` 에이전트(노드)에서 `print`로 출력되며 그 후 현재 메시지 수를 파악하기 위해 `print` 문을 추가합니다. 그리고 현재 상태를 `save_state` 함수로 저장합니다. 이 함수는 `utils.py` 파일에 따로 만들겠습니다.

```
(... 생략 ...)
# 터미널 창에서 사용자의 입력을 받고 graph를 실행(invoke)하는 부분
while True:
    user_input = input("\nUser\t: ").strip()

    if user_input.lower() in ['exit', 'quit', 'q']:
        print("Goodbye!")
        break

    state["messages"].append(HumanMessage(user_input))
    state = graph.invoke(state)

    print('\n----- MESSAGE COUNT\t', len(state["messages"]))

    save_state(current_path, state) # 현재 state 내용 저장
```

8. `save_state` 함수를 만들겠습니다. `book_writer.py` 파일에 작성할 수도 있지만 파일이 너무 커지는 것을 방지하기 위해 새로운 파일인 `utils.py`을 같은 폴더 내에 만들어 작성하겠습니다. `save_state` 함수는 `current_path`와 `state`를 매개변수로 받아 `current_path/data` 폴더에 `state`를 JSON 파일로 저장합니다. 현재 `state`에는 사용자와 AI 간의 대화 내용이 담긴 `messages`만 존재하므로 `[(m.__class__.__name__, m.content)]` 형태로 자료형과 대화 내용을 튜플로 변환하여 저장합니다. 이때 `ensure_ascii=False`와 `encoding='utf-8'`을 지정하여 저장된 JSON 파일의 한글이 깨지지 않도록 합니다.

파일을 저장하는 `save_state` 함수 만들기

utils.py

```
import os
import json

def save_state(current_path, state):
    if not os.path.exists(f"{current_path}/data"):
        os.makedirs(f"{current_path}/data")

    state_dict = {}

    messages = [(m.__class__.__name__, m.content) for m in state["messages"]]
    state_dict["messages"] = messages

    with open(f"{current_path}/data/state.json", "w", encoding='utf-8') as f:
        json.dump(state_dict, f, indent=4, ensure_ascii=False)
```

이 코드를 실행하면 다음처럼 터미널 창에서 대화할 수 있습니다. 아직 다른 기능은 구현되지 않았고 단순한 대화만 가능한 상태입니다. HYBE와 JYP의 경영 전략과 기업 문화에 대한 책을 써달라고 요청했지만 아직 알아서 진행하지는 못합니다.

User: HYBE와 JYP의 경영 전략과 기업 문화에 대한 책을 써줘.

===== COMMUNICATOR =====

AI: 안녕하세요! HYBE와 JYP의 경영 전략과 기업 문화에 대한 책을 요청해 주셨군요. 현재 AI팀은 이 주제에 대한 자료 수집 및 분석을 진행 중입니다. 두 회사의 경영 철학, 전략적 결정, 그리고 기업 문화를 비교하면서 독자들이 이해하기 쉽게 설명할 계획입니다.

혹시 이 책에서 특별히 다루었으면 하는 구체적인 내용이나 질문이 있으신가요? 또는 이 주제와 관련하여 궁금한 점이 있다면 말씀해 주세요. 여러분의 의견을 반영하여 더욱 유익한 책을 만들어 나가겠습니다.

----- MESSAGE COUNT 3

User: 알아서 써줘

===== COMMUNICATOR =====

AI: 알겠습니다! 요청해 주신 대로 HYBE와 JYP의 경영 전략 및 기업 문화에 대한 책을 저희 AI팀의 연구를 바탕으로 작성해보겠습니다. 두 회사의 역사와 성장 과정, 경영 철학, 그리고 각각의 독특한 기업 문화를 심도 있게 분석하여 독자들이 이해하기 쉽게 전달할 예정입니다.

또한, 두 회사의 최신 동향과 미래 전략에 대한 전망도 포함하여 독자들이 현재와 미래의 엔터테인먼트 산업을 이해하는 데 도움을 줄 수 있도록 할 것입니다. 만약 책이 완성되면 여러분에게 알릴 수 있도록 진행 상황을 지속적으로 업데이트하겠습니다.

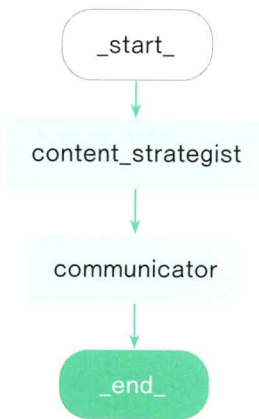
혹시 생각이 바뀌시거나 추가적으로 포함하고 싶은 내용이 생기시면 언제든지 말씀해 주세요. 여러분의 피드백은 저희에게 큰 도움이 됩니다. 감사합니다!

이 실행 결과는 data/state.json 파일에 저장됩니다. 시스템이 알아서 작업해 주면 좋겠지만 아직은 그렇지 못한 상태입니다. 이러한 결과물을 보고 언어 모델을 GPT-4o로 선택했는데도 그다지 쓸 만하지 않다고 생각해 버리는 사람이 많습니다. 하지만 이는 아직 우리가 적절한 가이드를 제공하지 못했기 때문일 수 있습니다.

책의 목차를 작성하는 콘텐츠 전략가 에이전트

보고서나 책을 쓰는 방법은 여러 가지 있겠지만 저는 목차를 먼저 만들고 나서 파트별로 어떤 내용을 넣을지 구상합니다. AI 에이전트도 이런 방식으로 작업을 진행하도록 구현해 보겠습니다. 물론 앞에서 만든 커뮤니케이터 에이전트에게 목차를 작성하라고 요청할 수도 있지만 앞으로 특정 기능을 독립적으로 수행하는 여러 AI 에이전트를 만들 계획입니다. 이 실습에서는 목차 작성을 전문으로 하는 콘텐츠 전략가 에이전트 content_strategist를 만들어 보겠습니다.

하나의 에이전트에게 하나의 임무만 맡기면 엉뚱하게 답변할 확률을 줄일 수 있습니다. 작업 결과가 마음에 들지 않을 때 어떤 에이전트를 개선해야 할지 파악하기도 쉽고요. 다음 그림처럼 그래프를 만들면 사용자가 입력한 메시지를 콘텐츠 전략가 에이전트 content_strategist가 우선 받고, 생성한 결과를 커뮤니케이터 에이전트 communicator가 전달받아서 사용자에게 보고하게 됩니다.



content_strategist를 추가한 그래프의 구조

Do it! 실습 목차를 작성하는 콘텐츠 전략가 에이전트 content_strategist 만들기

■ 결과 파일: sec01/book_writer_1.py, utils_1.py

1. 이번에 만드는 AI 에이전트 노드는 목차를 작성하는 콘텐츠 전략가를 의미하는 content_strategist입니다.

목차를 작성하는 content_strategist 에이전트 만들기

■ book_writer.py

```

from langgraph.graph import StateGraph, START, END
from langchain_openai import ChatOpenAI
from langchain_core.messages import AnyMessage, SystemMessage, HumanMessage, AIMessage
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers.string import StrOutputParser
from typing_extensions import TypedDict
from typing import List

from utils import save_state, get_outline, save_outline—4, 7

(... 생략 ...)

# 상태 정의
class State(TypedDict):
    messages: List[AnyMessage | str]

def content_strategist(state: State):
    print("\n\n===== CONTENT STRATEGIST =====")—1
  
```

```

content_strategist_system_prompt = PromptTemplate.from_template(
    """
    너는 책을 쓰는 AI 팀의 콘텐츠 전략가(Content Strategist)로서,
    이전 대화 내용을 바탕으로 사용자의 요구 사항을 분석하고, AI팀이 쓸 책의 세부 목차를 결정한다.

    지난 목차가 있다면 그 버전을 사용자의 요구에 맞게 수정하고, 없다면 새로운 목차를 제안한다.

    -----
    - 지난 목차: {outline}
    -----
    - 이전 대화 내용: {messages}
    """
)

content_strategist_chain = content_strategist_system_prompt | llm | StrOutputParser()

messages = state["messages"] # 상태에서 메시지 가져오기
outline = get_outline(current_path)

inputs = {
    "messages": messages,
    "outline": outline
}

gathered = ''
for chunk in content_strategist_chain.stream(inputs):
    gathered += chunk
    print(chunk, end='')

print()

save_outline(current_path, gathered)

content_strategist_message = f"[Content Strategist] 목차 작성 완료: {gathered}"
print(content_strategist_message)
messages.append(AIMessage(content_strategist_message))

return {"messages": messages}

def communicator(state: State):
    (... 생략 ...)

# 상태 그래프 정의
graph_builder = StateGraph(State)

```

```
# Nodes
graph_builder.add_node("communicator", communicator)
graph_builder.add_node("content_strategist", content_strategist)

# Edges
graph_builder.add_edge(START, "content_strategist")
graph_builder.add_edge("content_strategist", "communicator")
graph_builder.add_edge("communicator", END)

graph = graph_builder.compile()

(... 생략 ...)
```

- ❶ content_strategist 에이전트는 목차를 생성하고 ❷에서 save_outline() 함수를 이용해 생성한 목차를 저장하는 역할을 합니다. State에서 관리해도 되지만 파일로 저장하면 작업 과정을 파악하기 좋고 파일을 직접 수정할 수도 있습니다.
- ❷ content_strategist 에이전트를 위한 시스템 프롬프트를 정의합니다. 지난 목차(outline)와 이전 대화 내용(messages)이 주어지면 이전 대화 내용을 바탕으로 새로운 목차를 생성하라는 문구를 추가합니다. 실행 결과를 보면서 더 적절한 프롬프트로 만들어 나가겠습니다.
- ❸ 프롬프트와 언어 모델, 그리고 StrOutputParser를 연결한 체인을 만듭니다. 나중에 이 체인에 outline과 messages를 담아서 넣으면 되겠죠.
- ❹ get_outline 함수를 사용합니다. 이 함수는 utils.py파일에 구현할 예정입니다. content_strategist는 content_strategist_chain을 이용해 만든 목차 내용을 save_outline 함수를 이용해 저장합니다. 만약 저장되어 있던 목차(outline)가 이미 있다면 get_outline 함수로 읽어 올 수 있습니다.
- ❺ content_strategist_chain에 필요한 messages와 outline을 inputs로 만듭니다.
- ❻ 스트림 방식으로 새로운 목차(outline)를 생성해서 gathered에 담습니다.
- ❼ 완료된 이후에는 gathered에 담긴 목차를 save_outline 함수를 이용해 저장합니다. 이 save_outline 함수는 utils.py 파일에 곧 구현할 예정입니다.
- ❽ 현재 작업이 어떻게 끝났는지를 다음 노드인 communicator에 전달하기 위해 작업 결과를 AIMessage 인스턴스로 만들어 messages에 추가합니다.
- ❾ 대화 히스토리인 messages에 결과가 있으므로 communicator는 업데이트된 messages로 임무를 수행하게 됩니다. 따라서 현재의 진행 상황을 사용자에게 제대로 보고할 수 있습니다.
- ❿ 이제 새로 만든 노드를 그래프에 등록하고 노드간 연결 관계도 앞서 본 그래프와 같이 커뮤니케이터 에이전트인 communicator 앞에 연결되도록 수정합니다.

2. utils.py에 get_outline 함수와 save_outline 함수를 추가하겠습니다.

목차를 파일로 저장하는 함수 추가하기

utils.py

```
import os
import json

def save_state(current_path, state):
    (... 생략 ...)
```

```
def get_outline(current_path):
    outline = '아직 작성된 목차가 없습니다.'

    if os.path.exists(f"{current_path}/data/outline.md"):
        with open(f"{current_path}/data/outline.md", "r", encoding='utf-8') as f:
            outline = f.read()
    return outline
```

```
def save_outline(current_path, outline):
    if not os.path.exists(f"{current_path}/data"):
        os.makedirs(f"{current_path}/data")

    with open(f"{current_path}/data/outline.md", "w", encoding='utf-8') as f:
        f.write(outline)
    return outline
```

- 1 get_outline 함수의 매개변수는 current_path뿐입니다. 만약 current_path/data 폴더에 outline.md 파일이 있으면 그 파일 내용을 읽어서 반환합니다. 없는 경우에 대비해서 outline 변수에 '아직 작성된 목차가 없습니다.'라는 기본 문구를 설정해 두었으므로 파일이 없다면 이 문구가 반환됩니다.
- 2 save_outline 함수는 current_path와 outline을 매개변수로 받습니다. 매개변수로 받은 outline의 값을 current_path/data 폴더에 outline.md라는 파일명으로 저장합니다.

이제 대화를 통해 목차를 작성할 수 있습니다. 이 코드를 실행해 보면 목차를 작성하고 어떻게 수정하면 좋을지 물어봅니다. 목차를 작성하기 위해 현재 2개의 에이전트가 일을 하고 있습니다. 사용자가 원하는 내용을 입력하면 콘텐츠 전략가 에이전트 content_strategist가 목차를 작성하고 커뮤니케이터 에이전트 communicator가 진행 상황을 사용자에게 보고합니다.

User: JYP와 HYBE의 경영 전략과 기업 문화를 비교하는 책을 쓰고 싶어

===== CONTENT STRATEGIST =====

JYP와 HYBE의 경영 전략과 기업 문화를 비교하는 책에 대한 목차를 제안드리겠습니다. 이 책은 두 회사의 고유한 전략과 문화를 탐구하고, 그 차이점과 공통점을 분석하는 데 중점을 둘 것입니다.

제안 목차

1. **서문**

- 책의 목적과 중요성
- 연구 방법론 소개

2. **JYP 엔터테인먼트 개요**

- 회사 역사와 발전 과정
- 주요 아티스트와 성공 사례
- 비전과 미션

(... 생략 ...)

10. **결론**

- 주요 발견 요약
- 향후 연구 제안

11. **부록**

- 인터뷰 및 설문 자료
- 참고 문헌

이 목차는 JYP와 HYBE의 경영 전략과 기업 문화를 심도 있게 비교 분석하며, 독자에게 두 회사가 엔터테인먼트 산업에서 어떻게 독특한 위치를 차지하고 있는지를 이해시키는 데 초점을 맞추고 있습니다.

[Content Strategist] 목차 작성 완료: JYP와 HYBE의 경영 전략과 기업 문화를 비교하는 책에 대한 목차를 제안드리겠습니다. 이 책은 두 회사의 고유한 전략과 문화를 탐구하고, 그 차이점과 공통점을 분석하는 데 중점을 둘 것입니다.

===== COMMUNICATOR =====

AI: 안녕하세요! AI 작가팀의 커뮤니케이터입니다. 방금 전 AI팀의 콘텐츠 전략가가 JYP와 HYBE의 경영 전략과 기업 문화를 비교하는 책의 초기 목차를 완료했습니다. 목차는 사용자의 요구를 반영하여 두 기업의 주요 특징과 경영 철학을 깊이 있게 분석하고 있습니다.

현재 제안된 목차는 다음과 같습니다:

1. **서문**

2. **JYP 엔터테인먼트 개요**

(... 생략 ...)

7. **결론**

8. **부록**

현재 communicator는 content_strategist가 만든 목차를 한 번 더 출력하여 사용자에게 알려 주고 있습니다. 그런데 사용자는 content_strategist가 목차를 출력할 때 이미 터미널 창에서 확인할 수 있고 outline.md 파일을 열어서 확인할 수도 있으므로 굳이 communicator가 똑 같은 내용을 다시 사용자에게 알려 줄 필요는 없습니다.

3. content_strategist가 현재 목차를 메시지에 추가하지 않고 communicator도 사용자에게 목차를 출력하지 않도록 수정합니다.

목차를 중복 출력하지 않도록 수정하기

book_writer.py

```
# 목차를 작성하는 노드(agent)
def content_strategist(state: State):
    (... 생략 ...)
    # 메시지 추가
    content_strategist_message = f"[Content Strategist] 목차 작성 완료: {gathered}"
    print(content_strategist_message)
    messages.append(AIMessage(content_strategist_message))

    return {"messages": messages}

def communicator(state: State):
    # 사용자와 대화할 노드(agent): communicator
    print("\n\n===== COMMUNICATOR =====")

    # 시스템 프롬프트 정의
    communicator_system_prompt = PromptTemplate.from_template(
        """
        너는 책을 쓰는 AI 팀의 커뮤니케이터로서,
        AI 팀의 진행 상황을 사용자에게 보고하고, 사용자의 의견을 파악하기 위해 대화를 나눈다.

        사용자도 outline(목차)을 이미 보고 있으므로, 다시 출력할 필요는 없다.

        messages: {messages}
        """
    )
    (... 생략 ...)
```

코드를 실행하면 content_strategist만 목차를 출력합니다.

User: JYP와 HYBE의 경영 전략과 기업 문화를 비교하는 책을 쓰고 싶어

===== CONTENT STRATEGIST =====

사용자의 요구를 바탕으로 JYP와 HYBE의 경영 전략과 기업 문화를 비교하는 책의 목차를 이미 제안한 바 있습니다. 아래는 제안 된 목차입니다. 이 목차를 바탕으로 추가적인 요구사항이나 수정이 필요하다면 말씀해 주세요.

목차 제안:

1. **서문: K-POP 산업에서의 JYP와 HYBE**

- K-POP의 세계화
- JYP와 HYBE의 역할

(... 생략 ...)

[Content Strategist] 목차 작성 완료

===== COMMUNICATOR =====

AI: 지금까지의 진행 상황을 알려드리면, AI팀은 JYP와 HYBE의 경영 전략과 기업 문화를 비교하는 책의 목차를 작성했습니다. 이제 본격적인 집필 작업을 시작하려고 합니다. 사용자의 의견과 기대에 부합하는 방향으로 나아가기 위해 추가적으로 알고 싶으신 점이나 궁금한 부분이 있나요? 또는 강조하고 싶은 부분이나 특별히 다루고 싶은 주제가 있으면 말씀해 주세요. 이를 바탕으로 내용을 더 충실히 구성하겠습니다.

14-2 조장 역할을 하는 슈퍼바이저 에이전트

여러 사람이 협업할 때 조직 구조를 고려하듯이, 각각 전문 역할을 하는 AI 에이전트가 서로 유기적으로 협력하도록 만들 때에도 여러 전략을 구상할 수 있습니다. 가장 흔한 방법이 의사 결정을 하는 ‘장’을 뽑는 것입니다. 대학 때 조별 과제를 해도 조장을 뽑고 회사에서도 부서장을 세워서 누가 어떤 일을 어떻게 할지 판단하는 역할을 합니다. AI 에이전트들끼리 협력하도록 만들 때에도 이런 전략을 활용할 수 있습니다.

조장이 필요하다! — 슈퍼바이저 에이전트

14-1절에서 만든 프로그램에 보고서를 만들어 달라고 요청하지 않고 처음부터 ‘안녕?’이라고 입력하면 엉뚱하게 인사에 대한 목차를 생성합니다. 현재 그래프 구조는 사용자의 입력에 맞춰 무조건 목차를 생성하고 대화를 이어 나가게 되어 있기 때문입니다.

User: 안녕?

===== CONTENT STRATEGIST =====

안녕하세요! 책의 목차를 구성하기 위해 사용자 요구사항을 분석한 결과, 현재 제공된 정보가 제한적이지만, 일반적인 콘텐츠 전략에 따라 AI팀이 쓸 책의 초안 목차를 제안해 드리겠습니다. 이 목차는 AI가 책을 작성하면서 사용자가 원하는 내용을 충족할 수 있도록 가이드라인을 제공합니다.

제안 목차: AI와 책 쓰기

1. ****들어가며****
 - AI와 창작의 만남
 - 이 책의 목적과 대상 독자
2. ****AI의 역할 이해하기****
 - AI 작가의 개요
 - AI의 가능성과 한계

(... 생략 ...)

===== COMMUNICATOR =====

AI: 안녕하세요! AI팀의 커뮤니케이터입니다. 이미 목차 초안을 제안드렸지만, 사용자의 의견을 반영하여 더욱 맞춤형 내용을 만들고자 합니다. 제안된 목차에 대해 어떻게 생각하시나요? 추가하거나 수정하고 싶은 부분이 있으시면 말씀해 주세요. 사용자님의 의견을 반영하여 최종 책의 구성을 보다 완성도 있게 만들고 싶습니다. 어떤 부분이든 자유롭게 말씀해 주시면 감사하겠습니다!

따라서 사용자가 입력한 내용을 분석해 목차를 작성하는 콘텐츠 전략가 에이전트 `content_strategist`가 필요할지, 아니면 사용자와 소통하는 커뮤니케이터 에이전트 `communicator`가 필요할지 판단하는 조장이 필요합니다.

사용자의 요구 사항에 따라 어떤 AI 에이전트에게 일을 시킬지 판단하는 슈퍼바이저 에이전트 `supervisor`를 만들고, 이에 맞춰 그래프 구조를 바꾸겠습니다. 다음 그림과 같이 사용자가 메시지를 입력하면 `supervisor` 노드에서 목차를 쓰는 `content_strategist`에게 보낼지, 아니면 대화를 담당하는 `communicator`에게 바로 보낼지를 판단합니다. 목차를 작성하는 `content_strategist`로 가서 목차를 작성한 뒤에는 `communicator`로 넘어가서 결과를 사용자에게 보고하게 됩니다.



supervisor를 추가한 그래프의 구조

Do it! 실습 슈퍼바이저 에이전트 supervisor 추가하기

■ 결과 파일: sec02/book_writer_2.py

슈퍼바이저 에이전트 `supervisor`를 `book_wirter.py` 파일에 추가하겠습니다.

슈퍼바이저 에이전트 supervisor 추가하기

book_writer.py

(... 생략 ...)

모델 초기화

`llm = ChatOpenAI(model="gpt-4o")`


```

# 상태 정의
class State(TypedDict):
    messages: List[AnyMessage | str]
    task: str—①

def supervisor(state: State):—②
    print("\n\n===== SUPERVISOR =====")

    supervisor_system_prompt = PromptTemplate.from_template(
        """
        너는 AI 팀의 supervisor로서 AI 팀의 작업을 관리하고 지도한다.
        사용자가 원하는 책을 써야 한다는 최종 목표를 염두에 두고,
        사용자의 요구를 달성하기 위해 현재 해야 할 일이 무엇인지 결정한다.

        supervisor가 활용할 수 있는 agent는 다음과 같다.
        - content_strategist: 사용자의 요구 사항이 명확해졌을 때 사용한다. AI 팀의
        콘텐츠 전략을 결정하고, 전체 책의 목차(outline)를 작성한다.
        - communicator: AI 팀에서 해야 할 일을 스스로 판단할 수 없을 때 사용한다.
        사용자에게 진행 상황을 보고하고, 다음 지시를 물어본다.

        아래 내용을 고려하여, 현재 해야할 일이 무엇인지, 사용할 수 있는 agent를
        단답으로 말하라.

        -----
        previous_outline: {outline}
        -----
        messages:
        {messages}
        """
    )

    supervisor_chain = supervisor_system_prompt | llm | StrOutputParser()—④

    messages = state.get("messages", [])—⑤

    inputs = {
        "messages": messages,
        "outline": get_outline(current_path) }—⑥
    }

    task = supervisor_chain.invoke(inputs)—⑦

```

```

supervisor_message = AIMessage(f"[Supervisor] {task}")
messages.append(supervisor_message)
print(supervisor_message.content)

return {
    "messages": messages,
    "task": task
}

def supervisor_router(state: State):
    task = state['task']
    return task
(... 생략 ...)

# 상태 그래프 정의
graph_builder = StateGraph(State)

# 노드
graph_builder.add_node("supervisor", supervisor)
graph_builder.add_node("communicator", communicator)
graph_builder.add_node("content_strategist", content_strategist)

# 엣지
graph_builder.add_edge(START, "supervisor")
graph_builder.add_conditional_edges(
    "supervisor",
    supervisor_router,
    {
        "content_strategist": "content_strategist",
        "communicator": "communicator"
    }
)
graph_builder.add_edge("content_strategist", "communicator")
graph_builder.add_edge("communicator", END)

graph = graph_builder.compile()

# 그래프 도식화
graph.get_graph().draw_mermaid_png(output_file_path=absolute_path.replace('.py',
'.png'))

# 상태 초기화
state = State(
    messages = [

```

```

SystemMessage(
    f"""
        너희 AI들은 사용자의 요구에 맞는 책을 쓰는 작가 팀이다.
        사용자가 사용하는 언어로 대화하라.

        현재시각은 {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}이다.

        """
)
],
task="" ① task 추가
)

(... 생략 ...)

```

- ① State에 task를 문자열(str) 형식으로 추가해 상태를 다른 AI 에이전트들과 공유할 수 있게 합니다. task의 자료형은 나중에 수정할 예정입니다. 이렇게 설정했을 때의 불확실성을 살펴보기 위해 우선 문자열로 정의했습니다. 아래쪽에 state를 초기화하는 부분도 task=""로 초기화해 줍니다.
- ② supervisor를 새로 만듭니다. 이 에이전트는 다음에 할 일이 무엇인지 판단하는 역할을 합니다.
- ③ supervisor의 시스템 프롬프트입니다. 이 프롬프트에서는 content_strategist와 communicator를 선택하는 방법을 설명합니다. 입력값으로 받을 항목은 기존 목차를 의미하는 outline과 기존 대화 내용을 의미하는 messages입니다.
- ④ 체인을 연결하는 부분입니다. StrOutputParser로 content_strategist와 communicator 중에 하나가 나오면 supervisor_route에서 conditional_edge로 처리할 예정입니다.
- ⑤ state의 메시지를 가져오도록 설정합니다. 이렇게 설정하면 state에 'messages'가 있으면 그 값을 반환하고, 없으면 빈 리스트 []를 반환합니다. state['messages']로 설정해도 동일하지만 다양한 방식을 소개하기 위해 사용했습니다.
- ⑥ supervisor_chain에 넣을 inputs을 딕셔너리로 설정합니다.
- ⑦ supervisor_chain은 다음에 사용할 노드를 문자열 형태로 반환하므로 task 변수에 담습니다.
- ⑧ 이 task를 AIMessage 형태로 기존 messages에 추가하고 터미널 창에도 출력합니다.
- ⑨ 앞서 만든 supervisor의 결과를 이용해 노드 연결을 conditional_edge로 처리합니다. 현재 state에서 task의 값을 가져와 반환합니다.
- ⑩ 새로 만든 supervisor 노드를 그래프에 추가합니다.
- ⑪ 새로 추가한 supervisor 노드를 이용해 앞에서 살펴본 그래프와 같은 형태로 연결하기 위해 코드를 수정합니다.

이 코드를 실행하기 전에 기존에 생성된 ./data 폴더의 outline.md 파일과 state.json 파일을 삭제하고 시작해 보세요. 다음처럼 대화를 통해 목차를 만들 수 있습니다. ‘안녕?’이라고 입력해도 적절한 답변을 생성합니다.

```
User: 안녕?
===== SUPERVISOR =====
[Supervisor] communicator
===== COMMUNICATOR =====
AI: 안녕하세요! 책을 쓰는 AI팀의 커뮤니케이터입니다. 현재 저희 팀은 책의 각 장을 개발하고 있으며,
여러분의 피드백을 반영하여 내용을 더욱 개선하고 있습니다. 진행 상황이나 궁금한 점이 있으시면 언제든지
말씀해 주세요. 사용자의 의견은 저희에게 매우 중요합니다!
----- MESSAGE COUNT 4

(... 생략 ...)

User: HYBE와 JYP 비교하는 책을 써줘.
===== SUPERVISOR =====
[Supervisor] communicator
===== COMMUNICATOR =====
(... 생략 ...)
===== CONTENT STRATEGIST =====
(... 생략 ...)

### 제안 목차: HYBE와 JYP의 경영 전략과 기업 문화 비교

1. **서문**
   - 책의 목적과 배경
   - K-pop 산업에서 HYBE와 JYP의 중요성

(... 생략 ...)

8. **결론**
   - HYBE와 JYP의 경쟁력 요인 종합
   - 두 회사의 경영 전략과 문화에서 배울 점

이 목차는 경영 전략과 기업 문화에 초점을 맞추어 두 회사의 특징을 비교하고자 합니다. 추가적으로 다루고 싶으신 내용이나 수정이 필요하신 부분이 있으면 말씀해 주세요!
[Content Strategist] 목차 작성 완료

===== COMMUNICATOR =====
```


AI: 안녕하세요! 저희 AI팀은 HYBE와 JYP의 경영 전략과 기업 문화를 비교하는 책을 준비하고 있습니다. 목차는 이미 개발되어 있으니, 특정 부분이나 추가로 궁금한 점이 있으시면 언제든지 말씀해 주세요. 여러분의 피드백은 저희에게 많은 도움이 됩니다! 특별히 다루고 싶은 주제나 깊이 있게 알고 싶은 부분이 있으신가요?

----- MESSAGE COUNT 17

하지만 완성한 챗봇을 실행하면 다음과 같은 오류가 발생하는 경우가 있습니다.

KeyError: '현재 해야 할 일은 **content_strategist**입니다. 사용자의 요구에 맞는 책의 목차를 기반으로 전체 콘텐츠 전략을 결정하고, 필요한 경우 추가적인 내용이나 수정 사항을 반영하여 전체적인 아웃라인을 보완해야 합니다.'

특히 언어 모델을 GPT-4o-mini로 다운그레이드했을 때 이 오류가 더 많이 발생하는데, 그 이유는 supervisor_chain의 결과가 'content_strategist'나 'communicator'로 정확히 그 문자만 출력되지 않고 불필요한 문장까지 포함되기 때문입니다. 이어지는 실습에서 이 문제를 해결해 보겠습니다.

Do it! 실습 파이단틱의 BaseModel로 출력 형태 정의하기

■ 결과 파일: sec02/book_writer_3.py, models.py, utils_3.py

불필요한 문장까지 출력되는 문제를 개선하기 위해 08-3절에서 파이단틱 Pydantic의 BaseModel을 활용했습니다. 여기서도 supervisor_chain의 출력을 파이단틱 BaseModel을 활용한 클래스로 만들어 보겠습니다. 이렇게 하면 우리가 원하는 형태로 'content_strategist'나 'communicator'처럼 지정된 표현만 반환할 수 있습니다.

1. 새로운 파일 models.py를 만들고 다음처럼 Task 모델을 생성하는 코드를 작성합니다.

Task 모델 생성하기

models.py

```
from pydantic import BaseModel, Field
from typing import Literal

class Task(BaseModel):
    agent: Literal[
        "content_strategist",
        "communicator",
    ]
```

1

```

] = Field(
    ...,
    description=""
    작업을 수행하는 agent의 종류.
    - content_strategist: 콘텐츠 전략을 수립하는 작업을 수행한다. 사용자의 요구
사항이 명확해졌을 때 사용한다. AI 팀의 콘텐츠 전략을 결정하고, 전체 책의 목차(outline)를 작성한다.
    - communicator: AI 팀에서 해야 할 일을 스스로 판단할 수 없을 때 사용한다.
사용자에게 진행 상황을 보고하고, 다음 지시를 물어본다.
    """)

)

done: bool = Field(..., description="종료 여부")
description: str = Field(..., description="어떤 작업을 해야 하는지에 대한 설명")

done_at: str = Field(..., description="할 일이 완료된 날짜와 시간")

def to_dict(self):
    return {
        "agent": self.agent,
        "done": self.done,
        "description": self.description,
        "done_at": self.done_at
    }

```

- ❶ Task 클래스는 agent, done, description, done_at의 4개 필드가 있습니다. 이전 실습에서 supervisor_chain을 StrOutputParser로 지정했을 때는 원하는 문자만 출력하도록 강제하지 못했습니다. 우리가 원했던 결과는 content_strategist 또는 communicator 중에서 하나의 단어만 출력하는 것입니다. 이런 경우 agent 필드처럼 Literal로 지정하고 원하는 문구를 써주면 됩니다. 그리고 이에 대한 설명을 description에 작성합니다.
- ❷ done은 이 일을 끝냈는지 여부를 bool로 기록하는 필드입니다. 그리고 description 필드는 Task가 어떤 종류의 일인지 설명합니다. description 필드는 위에서 보겠지만 다음 노트, 즉 에이전트가 어떤 방식으로 일을 해야 할지 기록해 두므로 전체 흐름에서 중요한 역할을 합니다.
- ❸ done_at은 작업이 종료된 시각을 저장할 문자열 필드입니다.
- ❹ to_dict는 이 Task를 파일에 저장할 때 사용하기 위해 작성한 함수입니다.

2. 원래 작업하던 book_writer.py 파일에서 Task를 적용합니다.

Task 추가하기

book_writer.py

```
(... 생략 ...)
from utils import save_state, get_outline, save_outline
from models import Task ①
(... 생략 ...)

# 모델 초기화
llm = ChatOpenAI(model="gpt-4o")

# 상태 정의
class State(TypedDict):
    messages: List[AnyMessage | str]
    task_history: List[Task] ②

def supervisor(state: State):
    print("\n\n===== SUPERVISOR =====")

    # 시스템 프롬프트 정의
    supervisor_system_prompt = PromptTemplate.from_template(
        """
        (... 생략 ...)
        """
    )

    supervisor_chain = supervisor_system_prompt | llm.with_structured_output(Task) ③

    messages = state.get("messages", [])

    inputs = {
        "messages": messages,
        "outline": get_outline(current_path)
    }

    task = supervisor_chain.invoke(inputs)
    task_history = state.get("task_history", [])
    task_history.append(task) ④

    supervisor_message = AIMessage(f"[Supervisor] {task}")
    messages.append(supervisor_message)
    print(supervisor_message.content)

    return {
```

```

    "messages": messages,
    "task_history": task_history — 5
}

```

슈퍼바이저 라우트

```

def supervisor_router(state: State): — 6
    task = state['task_history'][-1]
    return task.agent
(... 생략 ...)

```

- 1 앞에서 models.py 파일에 만든 Task를 임포트합니다.
- 2 Task 목록을 저장할 task_history를 State에 추가합니다. 이곳에 현재 작업하는 내용을 리스트로 추가해 나갈 것입니다.
- 3 이전에 사용하던 StrOutputParser를 없애고 llm.with_structured_output(Task)로 교체합니다. 이렇게 하면 models.py 파일에서 정의한 Task의 필드에 맞춰 내용을 생성하도록 체인을 만들 수 있습니다.
- 4 이제 supervisor_chain.invoke()로 생성된 결과는 Task의 형태를 따릅니다. task 인스턴스를 state의 task_history에 추가합니다. supervisor의 해야 할 일 목록에 작업을 추가하는 개념이라고 생각하면 됩니다.
- 5 노드의 끝에서 딕셔너리 형태로 반환하면 state가 업데이트됩니다. 여기서는 task_history도 업데이트하도록 수정합니다.
- 6 supervisors_route는 supervisor 노드에서 만든 task에 따라 communicator 노드나 content_strategist 노드로 연결할지 판단하는 역할을 합니다. 가장 최근의 task가 무엇인지 판단하면 되므로 task_history의 맨 마지막 요소인 agent 필드값을 사용하면 Task 클래스에서 정의한 communicator와 content_strategist 중에 하나의 문자열이 정확히 담겨 반환됩니다.

3. 이제 content_strategist에 Task를 적용합니다.

목차 에이전트인 content_strategist 노드에 Task 적용하기

book_writer.py

```

(... 생략 ...)
# 목차를 작성하는 노드(agent)
def content_strategist(state: State):
    print("\n\n===== CONTENT STRATEGIST =====")
    (... 생략 ...)

    save_outline(current_path, gathered) # 목차 저장

    # 메시지 추가
    content_strategist_message = f"[Content Strategist] 목차 작성 완료"
    print(content_strategist_message)
    messages.append(AIMessage(content_strategist_message))

```



```

task_history = state.get("task_history", []) # task_history 가져오기

if task_history[-1].agent != "content_strategist":
    raise ValueError(f"Content Strategist가 아닌 agent가 목차 작성을 시도하고
있습니다.\n {task_history[-1]}")

task_history[-1].done = True
task_history[-1].done_at = datetime.now().strftime('%Y-%m-%d %H:%M:%S')

new_task = Task(
    agent="communicator",
    done=False,
    description="AI 팀의 진행 상황을 사용자에게 보고하고, 사용자의 의견을 파악하기 위해
대화를 나눈다",
    done_at=""
)
task_history.append(new_task)

print(new_task)

return {
    "messages": messages,
    "task_history": task_history
}
( ... 생략 ... )

```

- ① state에서 task_history를 가져와서 최근 작업이 content_strategist가 맞는지 확인합니다. content_strategist가 맞다면 done을 True로 처리하고 완료된 시각 done_at을 추가합니다. 만약 최근 Task의 agent 필드가 content_strategist가 아닌 경우에는 raise로 터미널 창에 오류 메시지를 출력해 문제를 알 수 있게 합니다.
- ② 현재 그래프 구조는 content_strategist가 목차를 만들면 그 다음에 communicator에서 작업 결과를 사용자에게 보고합니다. 따라서 이 작업 내용을 새로운 Task로 만들어 task_history에 추가합니다.
- ③ 마지막으로 현재 상태를 업데이트합니다.

4. 사용자와 대화하는 에이전트인 communicator 노드를 발전시킵시다.

communicator 노드 수정하기

book_writer.py

```
def communicator(state: State):
    # 사용자와 대화할 노드(agent): communicator
    print("\n\n===== COMMUNICATOR =====")

    communicator_system_prompt = PromptTemplate.from_template(
        """
        너는 책을 쓰는 AI팀의 커뮤니케이터로서,
        AI 팀의 진행 상황을 사용자에게 보고하고, 사용자의 의견을 파악하기 위해 대화를 나눈다.

        사용자도 outline(목차)을 이미 보고 있으므로, 다시 출력할 필요는 없다.

        outline: {outline}
        -----
        messages: {messages}
        """
    )

    # 시스템 프롬프트와 모델을 연결
    system_chain = communicator_system_prompt | llm

    # 상태에서 메시지 가져오기
    messages = state["messages"]

    # 입력값 정의
    inputs = {
        "messages": messages,
        "outline": get_outline(current_path) # 현재 outline 추가
    }

    # 스트림되는 메시지를 출력하면서 gathered에 모으기
    gathered = None

    print('\nAI\t: ', end='')
    for chunk in system_chain.stream(inputs):
        print(chunk.content, end='')

        if gathered is None:
            gathered = chunk
        else:
            gathered += chunk
```

1

```

messages.append(gathered)

task_history = state.get("task_history", [])
if task_history[-1].agent != "communicator":
    raise ValueError(f"Communicator가 아닌 agent가 대화를 시도하고 있습니다.\n
{task_history[-1]}")

task_history[-1].done = True
task_history[-1].done_at = datetime.now().strftime('%Y-%m-%d %H:%M:%S')

return {
    "messages": messages,
    "task_history": task_history
}

# 상태 그래프 정의
(... 생략 ...)

```

- ① communicator의 시스템 프롬프트에 목차를 의미하는 outline을 추가합니다. 사용자와 대화할 때 communicator가 현재 목차 정보를 몰라서 엉뚱한 말을 하는 경우를 방지하기 위한 목적입니다.
- ② State의 task_history를 가져와 완료로 표시합니다. 앞서 수정한 content_strategist 노드의 내용과 유사합니다. communicator 이후에는 후속 노드가 없으므로 추가 작업을 설정하지 않아도 됩니다.

5. 마지막으로 상태 초기화하는 부분에 task_history=[]로 빈 리스트를 만들어 줍니다.

초기 상태(state)에 빈 task_history 추가하기

book_writer.py

```

# 상태 초기화
state = State(
    messages = [
        SystemMessage(
            f"""
            너희 AI들은 사용자의 요구에 맞는 책을 쓰는 작가 팀이다.
            사용자가 사용하는 언어로 대화하라.

            현재 시각은 {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}이다.

            """
        )
    ],
    task_history=[]
)

```

```
)

while True:
    user_input = input("\nUser\t: ").strip()

(... 생략 ...)
```

6. 현재 상태로도 잘 작동하지만 `utils.py` 파일의 `save_state` 함수에 `task_history`도 저장 되도록 코드를 한 줄 추가하겠습니다. 매개변수 `state`를 통해 받은 값을 `.to_dict()` 메서드를 이용해 딕셔너리로 만드는 내용을 추가합니다. 이렇게 하면 `./data/state.json` 파일에 작업 이력인 `task_history`까지 깔끔하게 저장됩니다.

task_history 저장할 수 있도록 추가하기

utils.py

```
(... 생략 ...)

def save_state(current_path, state):
    if not os.path.exists(f"{current_path}/data"):
        os.makedirs(f"{current_path}/data")

    state_dict = {}

    messages = [(m.__class__.__name__, m.content) for m in state["messages"]]

    state_dict["messages"] = messages
    state_dict["task_history"] = [task.to_dict() for task in state.get("task_history",
[[]])

    with open(f"{current_path}/data/state.json", "w", encoding='utf-8') as f:
        json.dump(state_dict, f, indent=4, ensure_ascii=False)

(... 생략 ...)
```

현재 상태에서 `book_wirter.py` 파일을 실행해 봅시다. 실수 없이 대화를 통해 목차를 잘 생성해 나갑니다. 실행하기 전에 `./data` 폴더 내에 이전에 생성된 `outline.md`와 `state.json`을 삭제해야 처음부터 실행할 수 있습니다.

User: 안녕?

===== SUPERVISOR =====

[Supervisor] agent='communicator' done=False description='사용자의 요구사항이 명확하지 않으므로 진행 방향을 물어봐야 한다.' done_at=''

===== COMMUNICATOR =====

AI: 안녕하세요! 저희 AI팀의 책 작성 진행상황을 알려드릴게요. 현재 사용자의 요구사항이 명확하지 않아서 어떤 방향으로 책을 작성할지 고민 중입니다. 혹시 이 책에서 특별히 다뤘으면 하는 주제나 아이디어가 있을까요? 여러분의 의견이 큰 도움이 될 것입니다.

----- MESSAGE COUNT 4

User: HYBE와 JYP 비교하는 책을 써줘.

===== SUPERVISOR =====

[Supervisor] agent='content_strategist' done=False description='사용자가 요구하는 책의 주제(HYBE와 JYP 비교)가 명확해졌으므로 콘텐츠 전략을 결정하고, 전체 책의 목차 를 작성한다.' done_at=''

===== CONTENT STRATEGIST =====

사용자의 요구사항을 바탕으로, AI팀이 작성할 책의 목차를 결정했습니다. 이 책은 HYBE와 JYP, 두 주요 K-pop 엔터테인먼트 회사의 전반적인 비교와 분석을 다루게 됩니다. 다음은 제안하는 목차입니다:

서문

- K-pop 산업의 부상과 중요성
- HYBE와 JYP: K-pop의 두 거물

1장: 회사 개요

- HYBE 소개
 - 설립 역사
 - 주요 아티스트 및 업적
- JYP 소개
 - 설립 역사
 - 주요 아티스트 및 업적

(... 생략 ...)

7장: 업계 평가 및 미래 전망

- HYBE와 JYP의 현재 위치
- 향후 트렌드 및 도전 과제
- 두 회사의 미래 가능성

결론

- HYBE와 JYP의 비교 종합
- K-pop 산업 발전에 미친 영향

부록

- 인터뷰 및 기업 자료
- 추가 읽을거리

위 목적은 HYBE와 JYP의 다양한 측면을 철저히 분석하고 비교할 수 있도록 설계되었습니다. 추가로 심도 깊은 논의가 필요한 부분이나 수정이 필요한 부분이 있다면 언제든지 말씀해 주세요!

[Content Strategist] 목차 작성 완료

agent='communicator' done=False description='AI팀의 진행상황을 사용자에게 보고하고, 사용자의 의견을 파악하기 위한 대화를 나눈다' done_at=''

===== COMMUNICATOR =====

AI: 안녕하세요! 저희 AI팀은 사용자님의 요청에 따라 HYBE와 JYP를 비교 분석하는 책을 준비 중입니다. 현재 제안된 목적을 바탕으로 책의 구성을 계획하고 있으며, 두 회사의 다양한 측면을 심도 있게 다룰 예정입니다.

혹시 이 책에서 특별히 더 알고 싶으신 부분이 있거나 추가하고 싶은 주제가 있으신가요? 사용자님의 의견은 책 작성에 큰 도움이 됩니다. 감사합니다!

----- MESSAGE COUNT 8

User: quit

Goodbye!

./data/state.json 파일을 열어 보면 작업 이력도 잘 정리되어 있습니다.

```
{
  "messages": [
    [
      "SystemMessage",
      "\n\n      너희 AI들은 사용자의 요구에 맞는 책을 쓰는 작가팀이다.\n      사용자가 사용하는 언어로 대화하라.\n\n\n      현재시각은 2025-03-05 22:56:19이다.\n\n\n    ],
    [
      "HumanMessage",
      "안녕?"
    ],
    [
      "AIMessage",
      "[Supervisor] agent='communicator' done=False description='사용자의 요구사항이 명확하지 않으므로 진행 방향을 물어봐야 한다.' done_at=''"
    ],
    [
      "AIMessageChunk",
      "안녕하세요! 저희 AI팀의 책 작성 진행상황을 알려드릴게요. 현재 사용자의 요구사항이 명확하지 않아서 어떤 방향으로 책을 작성할지 고민 중입니다. 혹시 이 책에서 특별히 다뤘으면 하는 주제나 아이디어가 있을까요? 여러분의 의견이 큰 도움이 될 것입니다."
    ],
    [
```

```

        "HumanMessage",
        "HYBE와 JYP 비교하는 책을 써줘."
    ],
    [
        "AIMessage",
        "[Supervisor] agent='content_strategist' done=False description='사용자가  
요구하는 책의 주제(HYBE와 JYP 비교)가 명확해졌으므로 콘텐츠 전략을 결정하고, 전체 책의 목차를 작성  
한다.' done_at=''"
    ],
    [
        "AIMessage",
        "[Content Strategist] 목차 작성 완료"
    ],
    (... 생략 ...)
    "agent": "communicator",
    "done": true,
    "description": "AI팀의 진행상황을 사용자에게 보고하고, 사용자의 의견을 파악하기 위  
한 대화를 나눈다",
    "done_at": "2025-03-05 22:56:51"
    }
    ]
}

```


14-3 웹 검색과 RAG를 활용하는 벡터 검색 에이전트

14-2절에서 만든 AI 에이전트가 작성한 목차는 그럴싸하지만 확실한 근거가 없으므로 이대로 사용해도 될지 확인해야 합니다. 또한 더 좋은 책을 만들려면 참고 문헌을 찾아서 목차를 보완하는 과정도 필요합니다. 이때 인터넷 검색을 하거나 갖고 있는 책이나 문서를 이용할 수 있습니다. 이런 작업은 앞에서 이미 여러 차례 시켜 보았습니다. 인터넷 검색은 덕덕고 DuckDuckgo와 타빌리 Tavily를 활용하고, 기존에 갖고 있던 문서에서 쓸 만한 자료를 찾아 반영하는 작업도 앞에서 배운 RAG로 구현할 수 있습니다. 이 절에서는 웹 검색과 RAG를 활용하는 AI 에이전트를 개발해 보겠습니다.

벡터 DB를 활용해 효율적으로 웹 검색하기

10장에서는 덕덕고와 타빌리로 웹 검색 기능을 랭체인에 추가하는 방법을 배웠습니다. 질문을 받을 때마다 매번 인터넷을 검색해 답변하는 방식이었습니다. 그러나 이러한 방식은 책이나 보고서의 목차를 작성하는 데 적합하지 않습니다. 책이나 보고서의 목차를 작성하려면 웹 검색에서 찾은 문서의 전문을 읽고 내용을 파악해야 하는데 책을 쓸 때는 같은 문서 전체를 여러 번 검색해야 할 수 있습니다. 이때 타빌리를 사용하면 비용이 계속 발생하므로 같은 문서를 여러 번 검색하지 않도록 주의해야 합니다.

검색을 여러 번 하지 않고 검색한 내용을 모두 대화 내용에 담아 언어 모델이 찾아서 활용하게 할 수도 있습니다. 하지만 매번 대량의 텍스트를 언어 모델에 보내야 하므로 토큰을 많이 사용하게 됩니다. 이 방법은 비용이 많이 들 뿐만 아니라 답변 품질도 좋지 않을 수 있습니다. 사람에게 수십 페이지에 달하는 문서의 내용을 물어보면 제대로 대답하기 어려운 것과 마찬가지죠. 또한 사용하는 언어 모델의 컨텍스트 윈도우가 금방 한계에 도달해서 오류가 발생할 수도 있습니다.

이러한 문제를 해결하기 위해 웹에서 검색된 문서 전체를 벡터 DB에 저장하여 활용할 수 있습니다. 책 내용을 구성하는 데 필요한 질문이 있을 때 벡터 DB에서 RAG를 통해 검색하여 활용해 보겠습니다.

Do it! 실습 웹 검색 기능 만들기

결과 파일: sec03/tools_0.py

먼저 웹 검색 기능을 구현하고 WebBaseLoader를 활용해 웹 페이지의 전문을 읽어오는 방법을 알아보겠습니다. 그리고 검색 결과를 JSON 파일로 저장해 보겠습니다.

웹 검색을 위한 타빌리 검색 함수 만들기

타빌리 검색 기능을 활용해서 함수를 만들고 랭체인 도구로 사용하겠습니다.

현재 작업 중인 폴더에 새로운 파이썬 파일 tools.py를 생성하고 그 안에 다음과 같이 web_search 함수를 도구로 작성합니다.

타빌리 검색 함수를 활용해 웹 검색 기능 구현하기

tools.py

```
from tavily import TavilyClient
from langchain_core.tools import tool

@tool ❶
def web_search(query: str):
    """
    주어진 query에 대해 웹 검색을 하고 결과를 반환한다. ❷

    Args:
        query (str): 검색어

    Returns:
        dict: 검색 결과
    """
    client = TavilyClient()

    content = client.search(
        query,
        search_depth="advanced",
        include_raw_content=True, ❸
    )
    return content

if __name__ == "__main__":
    result = web_search.invoke("2025년 한국 영화 시장 전망") ❹
    print(result)
```

- 1 @tool 데코레이터는 함수를 랭체인이나 랭그래프에서 도구로 등록할 때 사용합니다. 이렇게 하면 해당 함수는 `.invoke()`를 통해 호출할 수 있게 됩니다.
- 2 이때 함수에 대한 설명을 작성해야 오류가 발생하지 않습니다. 여기에서는 '주어진 query에 대해 웹 검색을 하고 결과를 반환한다.'라고 간단한 설명을 추가합니다.
- 3 `include_raw_content`를 `True`로 설정하여 검색된 페이지의 전문을 가져오도록 만듭니다.
- 4 코드 하단에는 이 파일만 실행해도 테스트할 수 있도록 `__name__`으로 `main` 영역을 만들어 두었습니다. `web_search.invoke()` 안에 검색하고 싶은 내용을 입력하고 실행하면 됩니다.

이 코드를 실행하면 인터넷에서 검색한 결과가 출력됩니다. 반환된 결과는 딕셔너리 형태로 제공되며 검색 결과는 `results`에 리스트 형태로 담겨 있습니다. 각 페이지의 내용을 살펴보면 `raw_content`에 페이지 전문이 대부분 포함되어 있습니다. 그러나 일부 페이지에서는 `raw_content`가 `None`으로 되어 있습니다. 이 페이지들의 내용도 읽어 올 수 있다면 좋겠죠.

```
{'query': '2025년 한국 영화 시장 전망', 'follow_up_questions': None, 'answer': None,
'images': [], 'results': [{'url': 'https://eiec.kdi.re.kr/publish/columnView.do?cidx=15029&ccode=&pg=&sel_year=2025&sel_month=01', 'title': '영화시장 허리 붕괴 지속되나 드라마는 다소 회복할 전망 | 나라경제', 'content': '정리해 보자. 2025년 영화시장은 허리가 붕괴된 상황이 지속될 것으로 보인다. 해외 대작 출시가 이어지면서 2024년보다는 영화 관람객 수가 조금은 회복', 'score': 0.8732259, 'raw_content': None}
(... 생략 ...)
{'url': 'https://m.dailian.co.kr/news/view/1447721/?sc=sitemap', 'title': "봉준호의 '미키17'과 2025 한국영화 시장 판도 [D:영화 뷰] - 데일리안", 'content': '2025년, 한국 영화 산업이 새로운 전환점을 맞이한다. 대형 배급사들의 연간 라인업 축소, 중소 배급사의 적극적인 제작, 그리고 봉준호 감독의 신작', 'score': 0.8703443, 'raw_content': '공유하기\n봉준호의 \'미키17\'과 2025 한국영화
(... 생략 ...)}
```

WebBaseLoader로 페이지 전문 읽어 오기

타빌리 검색은 종종 페이지 내용을 제대로 읽어 오지 못합니다. 이런 경우에 10-1절에서 배운 `WebBaseLoader`를 활용하면 됩니다.

1. 타빌리 검색에서 URL을 제대로 읽어 올 수 있도록 `load_web_page` 함수를 만듭니다. `load_web_page` 함수가 잘 작동하는지 확인하기 위해 앞에서 작성한 테스트 코드를 주석 처리하고 방금 검색한 결과에서 `raw_content`가 `None`으로 나왔던 페이지를 골라 테스트합니다.

```
from tavily import TavilyClient
from langchain_core.tools import tool
from langchain_community.document_loaders import WebBaseLoader

@tool
def web_search(query: str):
    (... 생략 ...)
    return content

def load_web_page(url: str):
    loader = WebBaseLoader(url, verify_ssl=False)

    content = loader.load()

    return content

if __name__ == "__main__":
    # result = web_search.invoke("2025년 한국 영화 시장 전망") 주석으로 변경
    # print(result)

    result = load_web_page("https://eiec.kdi.re.kr/publish/columnView.do?cidx=15029&c-
code=&pp=20&pg=&sel_year=2025&sel_month=01")
    print(result)
```

이 코드를 실행해 보면 타빌리 검색에서 읽지 못하던 페이지를 잘 읽어 옵니다. **WebBaseLoader**는 기본적으로 여러 URL을 한 번에 읽을 수 있도록 설계되어 있지만 하나의 URL을 입력하면 단 하나의 페이지 정보만 담은 리스트 형태로 반환됩니다. 여기서 필요한 내용은 `page_content`입니다. 앞서 만든 `web_search` 도구에서 이 내용을 받아 와 앞에서 `None`으로 되어 있던 `raw_content`를 채우겠습니다.

Document(metadata={'source': 'https://eiec.kdi.re.kr/publish/columnView.do?cidx=15029&c-code=&pp=20&pg=&sel_year=2025&sel_month=01', 'title': '영화시장 허리 붕괴 지속되나 드라마는 다소 회복할 전망 | 나라경제 | KDI 경제교육·정보센터', 'language': 'ko'}, page_content='
(... 생략 ...)
전문가 기고 조영신.pdf\n\n\n\n\n\n\n\n\n\n\n2024년 12월 26일 전 세계인이 기다리던 <오징어 게임 시즌 2> 가 방영됐다. 방영 전부터 <오징어 게임>보다 훨씬 매력적인 작품이라는 평론가들의 평가 전해지면서 역사상 두 번째로 넷플릭스글로벌 1위를 기록할지도 모른다는 기대가 커지고 있었다.\n\r\n이렇게 이야기하면 사람들은 의아해한다.

(... 생략 ...)
)]

2. 타빌리 검색에서 가져오지 못한 페이지를 WebBaseLoader로 가져올 수 있도록 load_web_page 함수와 web_search 함수를 수정하겠습니다.

타빌리 검색에서 가져오지 못한 페이지는 WebBaseLoader로 가져오기

tools.py

```
from tavily import TavilyClient
from langchain_core.tools import tool
from langchain_community.document_loaders import WebBaseLoader

@tool
def web_search(query: str):
    """
    주어진 query에 대해 웹 검색을 하고 결과를 반환한다.

    Args:
        query (str): 검색어

    Returns:
        dict: 검색 결과
    """
    client = TavilyClient()

    content = client.search(
        query,
        search_depth="advanced",
        include_raw_content=True,
    )

    results = content["results"]

    for result in results:
        if result["raw_content"] is None:
            try:
                result["raw_content"] = load_web_page(result["url"])
            except Exception as e:
                print(f"Error loading page: {result['url']}")
                print(e)
                result["raw_content"] = result["content"]

    return results

def load_web_page(url: str):
    loader = WebBaseLoader(url, verify_ssl=False)

    content = loader.load()
```



```

raw_content = content[0].page_content.strip()—❶

while '\n\n\n' in raw_content or '\t\t\t' in raw_content:
    raw_content = raw_content.replace('\n\n\n', '\n\n')
    raw_content = raw_content.replace('\t\t\t', '\t\t')

return raw_content

if __name__ == "__main__":
    results = web_search.invoke("2025년 한국 영화 시장 전망")
    print(results[0])

# result = load_web_page("https://news.mt.co.kr/mtview.php?no=2024120402011362298")
# print(result)

```

- ❶ 먼저 페이지 내용만 반환하도록 load_web_page 함수를 수정합니다. WebBaseLoader는 여러 URL을 한 번에 처리하여 리스트로 반환하는데, 여기서는 URL을 하나씩 처리하므로 첫 번째 요소만 결과에서 꺼내 사용합니다. 이때 page_content에는 WebBaseLoader로 가져온 내용이 있으므로 page_content만 추출하고 앞뒤 공백을 제거하기 위해 strip()을 사용합니다. 또한 가져온 페이지 내용에 탭이나 줄 바꿈이 과도하게 포함될 수 있으므로 줄 바꿈이나 탭이 연속으로 3개 이상 있으면 while 문을 사용해 정리합니다. 이렇게 수정한 load_web_page 함수는 매개변수로 받은 URL의 페이지 내용을 문자열로 반환하게 됩니다.
- ❷ 이제 web_search 함수를 수정할 차례입니다. 타빌라 검색 결과로 받은 content는 딕셔너리 형태입니다. 그 중에 results 요소만 선택합니다. results에 들어 있는 항목 중에서 raw_content가 None인 경우에는 load_web_page 함수를 이용해 페이지 내용을 가져와 raw_content에 담습니다. 이때 일부 웹 사이트에서 load_web_page가 실패할 수 있으므로 try ~ except 문으로 오류를 처리합니다. 만약 load_web_page가 실패하면 content의 원래 값을 그대로 raw_content에 넣도록 처리합니다.

이 코드를 실행해 보면 raw_content가 None이었던 항목들도 load_web_page 함수를 통해 잘 읽어옵니다. 물론 웹 페이지에 오류가 있는 경우에는 읽어 올 수 없지만 타빌라 검색에서 해결되지 않던 웹 페이지 대부분은 처리됩니다. 이제 web_search의 실행 결과는 리스트 형태로 반환되며 그 안에는 검색된 웹 페이지의 정보가 딕셔너리 형태로 포함됩니다. web_search.invoke()로 검색된 첫 번째 항목을 살펴보면 다음과 같이 딕셔너리 형태로 되어 있음을 확인할 수 있습니다.

```

{'url': 'https://eiec.kdi.re.kr/publish/columnView.do?cidx=15029&ccode=&pp=20&pg=&sel_year=2025&sel_month=01', 'title': '영화시장 허리 붕괴 지속되나 드라마는 다소 회복할 전망 | 나라경제', 'content': '정리해 보자. 2025년 영화시장은 허리가 붕괴된 상황이 지속될 것으로 보인다. 해외 대작 출시가 이어지면서 2024년보다는 영화 관람객 수가 조금은 회복', 'score': 0.8732259, 'raw_content': '영화시장 허리 붕괴 지속되나 드라마는 다소 회복할 전망 | 나라경제 | KDI 경제교육·정보센터 (... 생략 ...)'}

```

웹 검색 결과 JSON 파일로 저장하기

이제 웹 검색 결과를 JSON 파일로 저장해 봅시다. 다음처럼 코드를 수정합니다.

웹 검색 결과를 JSON 파일로 저장하기

tools.py

```
from tavily import TavilyClient
from langchain_core.tools import tool
from langchain_community.document_loaders import WebBaseLoader
from datetime import datetime
import json
import os

absolute_path = os.path.abspath(__file__) # 현재 파일의 절대 경로 반환
current_path = os.path.dirname(absolute_path) # 현재 .py 파일이 있는 폴더 경로

@tool
def web_search(query: str):
    (... 생략 ...)
    for result in results:
        if result["raw_content"] is None:
            try:
                result["raw_content"] = load_web_page(result["url"])
            except Exception as e:
                print(f"Error loading page: {result['url']}")
                print(e)
                result["raw_content"] = result["content"]

    resources_json_path = f'{current_path}/data/resources_{datetime.now().strftime('%Y_%m%d_%H%M%S')}.json'
    with open(resources_json_path, 'w', encoding='utf-8') as f:
        json.dump(results, f, ensure_ascii=False, indent=4)

    return results, resources_json_path

def load_web_page(url: str):
    (... 생략 ...)
    return raw_content

if __name__ == "__main__":
    results, resources_json_path = web_search.invoke("2025년 한국 경제 전망")
    print(results)

    # result = load_web_page("https://news.mt.co.kr/mtview.php?no=2024120402011362298")
    # print(result)
```

- 1 우선 tools.py 파일이 있는 디렉터리 경로를 찾는 코드를 작성합니다. 이 코드는 14-1절에서 book_writer.py 파일에 작성한 내용과 동일합니다.
- 2 웹 검색 결과를 JSON 파일로 저장하는 코드입니다. 앞서 만든 resources_path를 이용해 JSON 파일을 저장할 폴더 경로를 찾고 파일명에 현재 시각을 추가하여 저장합니다. 여러분이 작업하는 폴더에 /data 폴더를 만들고 실행하세요.
- 3 web_search 함수가 검색 결과와 그 결과를 저장한 JSON 파일의 경로를 반환하도록 수정합니다. 메인 부분도 web_search의 결과를 results뿐만 아니라 resources_json_path와 함께 받도록 수정합니다.

작업하는 폴더에 /data 폴더를 만들고 코드에 '2025년 한국 경제 전망'을 입력한 후 실행하면 리스트 안에 딕셔너리가 담겨 있는 JSON 파일로 잘 저장됩니다.

```

chap15 > data > {} resources_2025_0207_172829.json > {} 2
1  [
2
3      {
4          "url": "https://www.hri.co.kr/upload/board/2887010266_WZR5c0kj_20241210123438.pdf",
5          "title": "[PDF] 2025년 한국경제 수정전망",
6          "content": "2025년 한국경제는 내수 경기 회복 지연 속 높은 외수 경기 불확실성으로 1.7% 정도의 성장세를",
7          "score": 0.945243,
8          "raw_content": "24-21(통권 979호) 2024.12.06. 2025년 한국경제 수정전망 - 성장경로 이탈이 우려되는 ?
9      },
10     {
11         "url": "https://biz.chosun.com/policy/policy_sub/2024/12/31/CI647NQHPVE5HEKV5SHAOLMV7KI/",
12         "title": "[2025 경제전망] 전문가 85% "내년 2% 성장 어렵다"... 저성장 직면 ...",
13         "content": "국내 경제 전문가 34명 중 85%가 넘는 29명이 내년 한국경제 성장률이 2%를 넘지 못할 것으로",
14         "score": 0.9407083,
15         "raw_content": "[2025 경제전망] 전문가 85% "내년 2% 성장 어렵다"... 저성장 직면한 대한민국 - 조선비즈
16     },
17     {
18         "url": "https://retailtalk.co.kr/Strategy/?bmode=view&idx=134588099",
19         "title": "2025 세계 경제 및 한국 경제 전망 : Strategy - 리테일톡, retailtalk",
20         "content": "2025 세계 경제 및 한국 경제 전망 : Strategy [5 min read]2025 세계 경제 및 한국 경제 전",
21         "score": 0.9094659,
22         "raw_content": "2025 세계 경제 및 한국 경제 전망 : Strategy\n
23     },
24     {
25         "url": "https://www.khan.co.kr/article/202411052101005",
26         "title": "2025년 한국경제 전망 - 경향신문",
27         "content": "얼마 전 발표된 2024년 3분기 경제성장률은 전분기 대비 0.1%로 한국경제가 경기침체 속에 여전",
28         "score": 0.9055316,
29         "raw_content": "전체 메뉴\n2025년 한국경제 전망\n최대 60자 이내로 입력하세요.\n얼마 전 발표된 2024년
30     },
31     {
32         "url": "https://eiec.kdi.re.kr/policy/domesticView.do?ac=0000191361",
33         "title": "2025년 거시경제 전망 | 국내연구자료 | KDI 경제교육·정보센터",
34         "content": "2025년 거시경제 전망 | 국내연구자료 | KDI 경제교육·정보센터 경제정책정보 발행물 경제교육",
35         "score": 0.8619225,
36         "raw_content": "2025년 거시경제 전망 | 국내연구자료 | KDI 경제교육·정보센터\n본문 내용으로 건너뛰기\
37 ]

```

이제 웹 검색을 하면 검색 결과가 JSON 파일로 저장됩니다. 앞서 설명했듯이 JSON 파일의 내용이 많아질수록 언어 모델을 활용해 원하는 답변을 받기 어려워지므로 검색 결과를 벡터 DB에 저장해서 활용하겠습니다.

웹 검색 결과를 랭체인 Document 객체로 변환하기

09장과 13장에서 RAG를 만들 때 PDF를 읽어 랭체인 Document 객체로 변환한 뒤, text splitter를 이용해 청크 단위로 나누고 이 청크들을 임베딩하여 벡터 DB에 저장했습니다. 이번에도 동일한 방법을 사용합니다. 다른 점은 이번에는 PDF가 아니라 JSON 형태로 저장된 웹 페이지 정보를 랭체인 Document 객체로 변환한다는 점입니다. 이를 위해 함수 2개를 사용할 것입니다.

1. 먼저 `web_page_to_document` 함수를 만듭니다. 이 함수는 하나의 웹 페이지 정보가 들어 오면 이를 랭체인 Document 객체로 변환하는 기능을 합니다.

웹 검색 결과를 Document 객체로 만들기

tools.py

```
from tavily import TavilyClient
from langchain_core.tools import tool
from langchain_community.document_loaders import WebBaseLoader
from langchain_core.documents import Document

from datetime import datetime
import json
import os
absolute_path = os.path.abspath(__file__) # 현재 파일의 절대 경로 반환
current_path = os.path.dirname(absolute_path) # 현재 .py 파일이 있는 폴더 경로

@tool
def web_search(query: str):
    (... 생략 ...)
    return results, resources_json_path

def web_page_to_document(web_page):
    if len(web_page['raw_content']) > len(web_page['content']):
        page_content = web_page['raw_content']
    else:
        page_content = web_page['content']
```



```

document = Document(
    page_content=page_content,
    metadata={
        'title': web_page['title'],
        'source': web_page['url']
    }
)

return document

```

(... 생략 ...)

- 1 타빌리 검색에서 페이지를 열기 전에 얻을 수 있는 페이지 정보는 content에 기록되고, 실제로 페이지를 열었을 때의 전문이 raw_content에 기록됩니다. 하지만 raw_content에 정보가 부족한 경우가 있습니다. 이는 최근에 웹 페이지 내용이 삭제되었거나 수정된 경우에 해당하는데, 이런 경우 raw_content와 content 중에 길이가 더 긴 정보를 Document의 page_content로 사용합니다.
- 2 웹 페이지의 정보를 랭체인의 Document로 변환하는 코드입니다. 랭체인에서 제공하는 Document 클래스는 기본적으로 벡터 검색에 활용할 수 있는 문서의 실제 내용인 page_content와 문서에 대한 추가 정보인 metadata를 갖고 있습니다. 앞선 실습의 결과를 보면 리스트에 각 페이지 정보가 있고 그 안에 title, url, content, score, raw_content 키에 해당하는 값이 포함되어 있습니다. page_content에 ①에서 정의한 page_content를, metadata에 웹 페이지의 title과 url을 대입하여 Document를 생성합니다. 그리고 document를 반환합니다.

2. 다음으로 web_page_json_to_documents 함수를 만듭니다. 이 함수는 json_file 경로를 입력받아 파일을 읽고 각 웹 페이지 정보를 web_page_to_document 함수에 하나씩 전달하여 Document 객체로 변환된 값을 받습니다. 변환된 Document들은 리스트에 차례대로 추가하여 반환합니다. 테스트를 위해 이 파일의 메인 영역에는 앞서 생성된 JSON 파일 경로를 web_page_json_to_documents에 추가합니다. 그리고 documents의 마지막 요소를 출력합니다.

JSON 파일에 저장된 정보를 Document 객체들로 변환하기

tools.py

```

(... 생략 ...)
def web_page_json_to_documents(json_file):
    with open(json_file, "r", encoding='utf-8') as f:
        resources = json.load(f)

    documents = []

    for web_page in resources:
        document = web_page_to_document(web_page)

```

```

        documents.append(document)

    return documents

def load_web_page(url: str):
    (... 생략 ...)
    return raw_content

if __name__ == "__main__":
    # results, resources_json_path = web_search.invoke("2025년 한국 경제 전망")
    # print(results)

    # result = load_web_page("https://news.mt.co.kr/mtview.php?no=2024120402011362298")
    # print(result)

    documents = web_page_json_to_documents(f'{current_path}/data/
resources_2024_1209_222930.json')
    파일명 확인
    print(documents[-1])

```

이 코드를 실행하면 다음처럼 변환된 Document 객체가 출력됩니다.

```

(... 생략 ...)
- 2025년 세계경제는 인플레이션의 안정과 통화정책 완화 등이 긍정적 요인으로 꼽히지만, 지역 분쟁을 비롯한 지정학적 불확실성과 통화정책 변화에 따른 금융시장 변동성, 글로벌 보호무역 기조 강화 등이 성장의 제약 요인으로 작용하면서 2024년 수준의 성장률이 예상됨.
(... 생략 ...)

```

Document 객체들을 청크 단위로 자르기

웹 페이지의 정보를 RAG 방식으로 활용하려면 청크 단위로 나누어야 합니다. 웹 페이지 정보를 랭체인 Document 객체로 변환하는 함수를 만들었으므로 09-2절에서 배운 RecursiveCharacterTextSplitter를 활용해서 청크 단위로 나눌 수 있습니다.

split_documents 함수는 여러 개의 documents를 매개변수로 받아 chunk_size와 chunk_overlap에 맞춰 청크 단위로 나누는 역할을 합니다. 09-2절에서 RAG를 사용한 코드와 거의 같으며 함수가 잘 작동하는지 확인하기 위해 print 문을 추가했습니다. 이 코드를 테스트하기 위해 main 부분에서 split_documents 함수를 호출합니다.

```

from tavily import TavilyClient
from langchain_core.tools import tool
from langchain_community.document_loaders import WebBaseLoader
from langchain_core.documents import Document
from langchain_text_splitters import RecursiveCharacterTextSplitter

(... 생략 ...)

def web_page_json_to_documents(json_file):
    (... 생략 ...)
    return documents

def split_documents(documents, chunk_size=1000, chunk_overlap=100):
    print('Splitting documents...')
    print(f"{len(documents)}개의 문서를 {chunk_size}자 크기로 중첩 {chunk_overlap}자로 분할
합니다.\n")

    text_splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size, chunk_overlap=chunk_overlap
    )

    splits = text_splitter.split_documents(documents)

    print(f"총 {len(splits)}개의 문서로 분할되었습니다.")
    return splits

def load_web_page(url: str):
    (... 생략 ...)
    return raw_content

if __name__ == "__main__":
    # results, resources_json_path = web_search.invoke("2025년 한국 경제 전망")
    # print(results)

    # # result = load_web_page("https://news.mt.co.kr/mtview.php?no=2024120402011362298")
    # # print(result)

    documents = web_page_json_to_documents(f'{current_path}/data/resources_2024_1210_0
00142.json')

    splits = split_documents(documents)
    print(splits)

```

이 코드를 실행해 보면 다음과 같이 청크 단위로 잘 나뉘었습니다.

```
Splitting documents...
5개의 문서를 1000자 크기로 중첩 100자로 분할합니다.

총 10개의 문서로 분할되었습니다.
(... 생략 ...)
```

벡터 DB에 저장하기

이제 청크를 벡터 DB에 저장하는 함수를 만듭시다. 09-2절에서 배운 것처럼 이번에도 RAG를 위한 벡터 DB 설정을 해야 합니다.

`documents_to_chroma` 함수를 만듭시다. 이 함수는 랭체인 `Document` 리스트를 매개변수로 받아서 `split_documents` 함수를 이용해 청크 단위로 나누고 크로마 DB에 저장하는 역할을 합니다. 웹 검색을 한 결과를 벡터 DB인 크로마 DB에 저장할 때 같은 웹 페이지가 다시 검색되면 벡터 DB에 결과가 중복으로 저장될 수 있습니다. 이렇게 되면 RAG로 벡터 검색을 할 때 여러 문서의 청크를 가져오지 못하고 동일한 청크를 가져올 수 있습니다. 이를 방지하기 위해 이미 벡터 DB에 저장된 URL은 중복으로 저장되지 않도록 처리합니다.

크로마 DB에 저장하기

tools.py

(... 생략 ...)

```
from datetime import datetime
import json
import os

absolute_path = os.path.abspath(__file__) # 현재 파일의 절대 경로 반환
current_path = os.path.dirname(absolute_path) # 현재 .py 파일이 있는 폴더 경로

from langchain_openai import OpenAIEmbeddings
from langchain_chroma import Chroma

# 오픈AI Embedding 설정
embedding = OpenAIEmbeddings(model='text-embedding-3-large')

# 크로마 DB 저장 경로 설정
persist_directory = f"{current_path}/data/chroma_store"

# Chroma 객체 생성
```



```

vectorstore = Chroma(
    persist_directory=persist_directory,
    embedding_function=embedding
)

@tool
def web_search(query: str):
    (... 생략 ...)

    return results

(... 생략 ...)
def split_documents(documents, chunk_size=1000, chunk_overlap=100):
    (... 생략 ...)
    return splits

def documents_to_chroma(documents, chunk_size=1000, chunk_overlap=100):
    print("Documents를 Chroma DB에 저장합니다.")

    urls = [document.metadata['source'] for document in documents]

    stored_metadatas = vectorstore._collection.get()['metadatas']
    stored_web_urls = [metadata['source'] for metadata in stored_metadatas]

    new_urls = set(urls) - set(stored_web_urls)

    new_documents = []

    for document in documents:
        if document.metadata['source'] in new_urls:
            new_documents.append(document)
            print(document.metadata)

    splits = split_documents(new_documents, chunk_size=chunk_size, chunk_overlap=chunk_overlap)

    if splits:
        vectorstore.add_documents(splits)
    else:
        print("No new urls to process")

def add_web_pages_json_to_chroma(json_file, chunk_size=1000, chunk_overlap=100):
    documents = web_page_json_to_documents(json_file)
    documents_to_chroma(
        documents,

```

```

        chunk_size=chunk_size,
        chunk_overlap=chunk_overlap
    )

def load_web_page(url: str):
    (... 생략 ...)
    return raw_content

if __name__ == "__main__":
    # results, resources_json_path = web_search.invoke("2025년 한국 경제 전망")
    # print(results)

    # # result = load_web_page("https://news.mt.co.kr/mtview.php?no=2024120402011362298")
    # # print(result)

    # documents = web_page_json_to_documents(f'{current_path}/data/resources_2024_1210_
000142.json')

    # splits = split_documents(documents)

    add_web_pages_json_to_chroma(f'{current_path}/data/resources_2024_1210_000142.json')

```

파일명 확인

9

- ❶ 임베딩은 OpenAIEmbeddings의 text-embedding-3-large를 선택하고 벡터 DB는 크로마 DB를 선택합니다. 크로마 DB가 저장될 폴더 위치를 설정하고 vectorstore 변수에 Chroma를 선언해 줍니다.
- ❷ 매개변수로 받은 documents의 metadata에 있는 source를 이용해 URL을 가져옵니다.
- ❸ 이미 벡터 DB에 저장된 문서들의 metadata에서 URL을 가져옵니다.
- ❹ 저장할 문서들과 이미 저장되어 있던 문서들의 URL은 set을 사용해 중복을 없애서 아직 벡터 DB에 저장되어 있지 않은 URL들을 골라냅니다.
- ❺ 새로운 URL에 해당하는 document들만 골라서 줍니다.
- ❻ 이 문서들을 split_documents 함수를 이용해 청크 단위로 자릅니다.
- ❼ 만약 새로 저장할 문서가 있다면 크로마 DB에 청크들을 저장합니다.
- ❽ JSON 파일에서 documents를 만들고 그 documents들을 크로마 DB에 저장합니다. 그리고 documents_to_chroma 함수를 실행시킵니다.
- ❹ 새로 만든 기능을 테스트하기 위해 이 코드의 메인 부분에서 기존에 만들었던 JSON 파일을 이용해 실행합니다.

터미널 창에는 다음과 같은 진행 상황이 출력되었습니다.

```
Documents를 Chroma DB에 저장합니다.
{'title': '[PDF] 2025년 한국경제 수정전망', 'source': 'https://www.hri.co.kr/upload/
board/2887010266_WZR5c0kj_20241210123438.pdf'}
{'title': '[2025 경제전망]① 전문가 85% "내년 2% 성장 어렵다"... 저성장 직면 ...', 'source':
'https://biz.chosun.com/policy/policy_sub/2024/12/31/CI647NQHPVE5HEKV5HA0LMV7KI/'}
{'title': '2025 세계 경제 및 한국 경제 전망 : Strategy - 리테일톡, retailtalk', 'source':
'https://retailtalk.co.kr/Strategy/?bmode=view&idx=134588099'}
{'title': '2025년 한국경제 전망 - 경향신문', 'source': 'https://www.khan.co.kr/article/
202411052101005'}
{'title': '2025년 거시경제 전망 | 국내연구자료 | KDI 경제교육·정보센터', 'source': 'https://
eiec.kdi.re.kr/policy/domesticView.do?ac=0000191361'}
Splitting documents...
5개의 문서를 1000자 크기로 중첩 100자로 분할합니다.
```

크로마 DB도 지정한 위치에 성공적으로 생성되었습니다.

```
▼ data
  ▼ chroma_store
    > b8c68f8c-e4cc-4131-b365-eacc820c940d
  chroma.sqlite3
```

리트리버 만들기

아직 벡터 DB에 몇 개의 문서만 저장되어 있지만 벡터 검색이 잘 작동하는지 확인해 보겠습니다.

벡터 검색을 확인하는 `retrieve` 함수를 만듭니다. 이 함수는 `query`와 `top_k`를 매개변수로 받습니다. `retrieve` 함수는 13장에서 만든 `rag_with_langgraph.ipynb`를 그대로 가져온 것이며, 사용자의 질문에 응답할 수 있도록 랭그래프에 연결하기 위해 `@tool` 데코레이터를 붙였습니다. 메인 부분에는 리트리버를 테스트하는 코드를 추가합니다.

```
리트리버 만들기 tools.py

(... 생략 ...)

def load_web_page(url: str):
    (... 생략 ...)

    return raw_content
```

```

@tool
def retrieve(query: str, top_k: int=5):
    """
    주어진 query에 대해 벡터 검색을 수행하고, 결과를 반환한다.
    """
    retriever = vectorstore.as_retriever(search_kwargs={"k": top_k})
    retrieved_docs = retriever.invoke(query)

    return retrieved_docs

if __name__ == "__main__":
    (... 생략 ...)

    retrieved_docs = retrieve.invoke({"query": "한국 경제 위험 요소"})
    print(retrieved_docs)

```

이 코드를 실행한 결과 다음과 같이 리트리버가 잘 작동하는 것을 확인할 수 있습니다. 아직 벡터 DB에 저장된 문서가 많지 않아 입력한 키워드와 연관성은 그다지 높지 않을 수 있지만, 웹 검색 후 벡터 DB에 저장하는 기능을 만들어 두었으니 점점 더 많은 자료를 기반으로 유용한 정보를 반환할 겁니다.

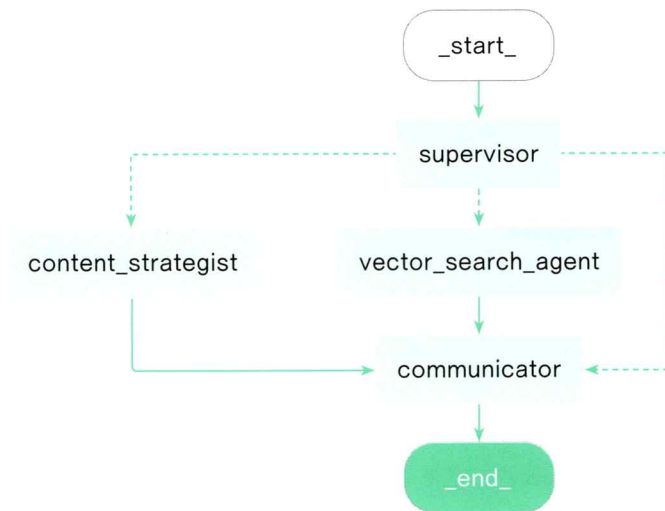
```

Document (id='0faa130c-779a-4201-a8de-f45de3188ac7', metadata={'source': 'https://www.hri.co.kr/upload/board/2887010266_WZR5c0kj_20241210123438.pdf', 'title': '[PDF] 2025년 한국경제 수정전망'}, page_content=' 주 : OECD의 선진국은 OECD 회원국, 신흥개도국은 비OECD 회원국임. 2-2. 국내 경기 : 확대되고 있는 성장경로 이탈 가능성 ○(전반적인 경기 활력 저하) 내수 부진이 이어지는 가운데 그동안 경제를 지탱했 던 수출의 성장 견인력 위축과 이로 인한 전반적인 경기 활력 저하 - 3분기 경제
(... 생략 ...)

```

관련 높은 청크 찾는 벡터 검색 에이전트

지금까지 인터넷 검색 후 검색 결과를 벡터 DB에 저장하는 기능과 RAG를 구현하는 기능을 개발했지만 아직 랭그래프로 연결하여 사용하지는 않았습니다. 벡터 DB에 저장한 내용에서 검색 내용과 관련이 높은 청크 문서를 가져오는 벡터 검색 에이전트 `vector_search_agent`를 추가하고 어떤 식으로 동작하는지 확인해 봅시다



vector_search_agent를 추가한 그래프의 구조

Do it! 실습 랭그래프에 연결하기

■ 결과 파일: sec03/book_writer.py, models.py

문서에서 필요한 청크를 찾아오는 벡터 검색 에이전트를 만들고 목차를 작성할 때 사용할 참고 자료를 JSON 파일로 저장해 보겠습니다.

비어 있는 벡터 검색 에이전트 만들기

1. 먼저 벡터 검색 에이전트 `vector_search_agent`에 그래프가 어떻게 작동하는지 확인하는 코드를 추가하겠습니다. 처음에는 오류 메시지가 출력될 수 있지만 이를 차차 해결하면서 프로그램을 완성해 봅시다.

비어 있는 RAG 에이전트 추가하기

■ book_writer.py

```
(... 생략 ...)
from models import Task
from tools import retrieve ①

from datetime import datetime
(... 생략 ...)

def supervisor(state: State):
    print("\n\n===== SUPERVISOR =====")
```

```

supervisor_system_prompt = PromptTemplate.from_template(
    """
    너는 AI 팀의 supervisor로서 AI 팀의 작업을 관리하고 지도한다.
    사용자가 원하는 책을 써야 한다는 최종 목표를 염두에 두고,
    사용자의 요구를 달성하기 위해 현재 해야 할 일이 무엇인지 결정한다.

    supervisor가 활용할 수 있는 agent는 다음과 같다.
    - content_strategist: 사용자의 요구 사항이 명확해졌을 때 사용한다. AI 팀의 콘텐츠 전략을
    결정하고, 전체 책의 목차(outline)를 작성한다.
    - communicator: AI 팀에서 해야 할 일을 스스로 판단할 수 없을 때 사용한다. 사용자에게 진행
    상황을 사용자에게 보고하고, 다음 지시를 물어본다.
    - vector_search_agent: 벡터 DB 검색을 통해 목차(outline) 작성에 필요한 정보를 확보한다.

    아래 내용을 고려하여, 현재 해야 할 일이 무엇인지, 사용할 수 있는 agent가 무엇인지 단답으로
    말하라.

    -----
    previous_outline: {outline}
    -----

    messages:
    {messages}
    """
)
(... 생략 ...)

return {
    "messages": messages,
    "task_history": task_history
}

# supervisor's route
def supervisor_router(state: State):
    task = state['task_history'][-1]
    return task.agent
(... 생략 ...)

def vector_search_agent(state: State):
    print("\n\n===== VECTOR SEARCH AGENT =====")

    task = state['task_history'][-1]

    print(task)

def content_strategist(state: State):
    (... 생략 ...)

```

```
(... 생략 ...)
```

```
# 상태 그래프 정의
graph_builder = StateGraph(State)

# 노드
graph_builder.add_node("supervisor", supervisor)
graph_builder.add_node("communicator", communicator)
graph_builder.add_node("content_strategist", content_strategist)
graph_builder.add_node("vector_search_agent", vector_search_agent) ④

# 엣지
graph_builder.add_edge(START, "supervisor")
graph_builder.add_conditional_edges(
    "supervisor",
    supervisor_router,
    {
        "content_strategist": "content_strategist",
        "communicator": "communicator",
        "vector_search_agent": "vector_search_agent" ④
    }
)
graph_builder.add_edge("content_strategist", "communicator")
graph_builder.add_edge("vector_search_agent", "communicator") ④
graph_builder.add_edge("communicator", END)

graph = graph_builder.compile()
(... 생략 ...)
```

- ❶ 지금까지 만든 tools.py 파일에 벡터 검색을 할 수 있는 retrieve 함수를 임포트합니다.
- ❷ 누가 어떤 일을 할지를 결정하는 슈퍼바이저 에이전트인 supervisor의 시스템 프롬프트에 벡터 검색 에이전트인 vector_search_agent를 선택할 수 있도록 작성합니다. 이제 supervisor가 vector_search_agent를 사용할 수 있음을 알게 되었지만, 실제로 vector_search_agent를 선택할 수 있게 하려면 supervisor_chain에서 사용하는 Task에도 vector_search_agent를 추가해야 합니다. 이 부분은 다음 단계에서 models.py 파일에 추가하겠습니다.
- ❸ vector_search_agent를 추가합니다. 아직 미완성 상태이지만 supervisor가 vector_search_agent에 일을 시킬 수 있는지 확인하기 위해 테스트로 '===== VECTOR SEARCH AGENT ====='를 출력합니다. 그리고 vector_search_agent를 실행하기 위해 supervisor가 어떤 식으로 Task를 생성하는지 확인할 수 있도록 Task를 출력합니다.

- 4 새로 만든 `vector_search_agent`를 그래프에 등록합니다. 노드를 추가하고 `supervisor`에서 `conditional_edge`를 사용하여 `vector_search_agent`도 활용할 수 있도록 했습니다. 벡터 검색을 수행한 다음에는 사용자에게 보고할 수 있도록 커뮤니케이터 에이전트인 `communicator`로 연결합니다.

2. `models.py` 파일에서 `Task`를 수정해 `vector_search_agent`를 선택할 수 있도록 설정합니다. `Literal`에 `vector_search_agent`를 추가하고 이에 대한 설명을 `description`에 적습니다.

Task에 rag_agent 추가하기

models.py

```
class Task(BaseModel):
    agent: Literal[
        "content_strategist",
        "communicator",
        "vector_search_agent",
    ] = Field(
        ...,
        description="""
        작업을 수행하는 agent의 종류.
        - content_strategist: 콘텐츠 전략을 수립하는 작업을 수행한다. 사용자의 요구 사항이 명확해졌을 때 사용한다. AI 팀의 콘텐츠 전략을 결정하고, 전체 책의 목차(outline)를 작성한다.
        - communicator: AI 팀에서 해야 할 일을 스스로 판단할 수 없을 때 사용한다. 사용자에게 진행 상황을 보고하고, 다음 지시를 물어본다.
        - vector_search_agent: 벡터 DB 검색을 통해 목차(outline) 작성에 필요한 정보를 확보한다.
        """
    )
```

`book_writer.py` 코드를 테스트하기 위해 `./data` 폴더에 있는 `outline.md` 파일이 있다면 삭제 후 실행합니다. 이제 슈퍼바이저 에이전트가 어떤 에이전트를 선택할 수 있으므로 목차를 먼저 생성할 수도 있고 RAG를 먼저 시도할 수도 있습니다. 이번 실행에서는 슈퍼바이저 에이전트가 처음부터 벡터 검색 에이전트로 필요한 자료를 수집하려 했습니다. 다만 현재 RAG 기능이 구현되어 있지 않아서 오류 메시지가

◆ 만약 벡터 검색 에이전트로 슈퍼바이저 에이전트가 `task`를 주지 않는다면 '참고 자료를 검색하자.'라고 입력해 다시 시도해보세요.

User: HYBE와 JYP 비교하는 책 쓰자

===== SUPERVISOR =====

```
[Supervisor] agent='vector_search_agent' done=False description='HYBE와 JYP에 대한 정보를 수집하여 목차 작성에 필요한 자료를 준비합니다.' done_at=''
```



```

===== VECTOR SEARCH AGENT =====
agent='vector_search_agent' done=False description='HYBE와 JYP에 대한 정보를 수집하여 목차
작성에 필요한 자료를 준비합니다.' done_at=''

===== COMMUNICATOR =====

AI: 안녕하세요! 현재 우리는 HYBE와 JYP를 비교하는 책을 작성하기 위해 정보 수집 단계에 있습니다. 목
차는 아직 작성되지 않았지만, 두 회사의 주요 특징과 차별점을 중심으로 구성을 계획 중입니다. 사용자께
서 생각하시는 흥미로운 비교 포인트나 추가적으로 다루었으
면 하는 주제가 있다면 말씀해 주세요. 여러분의 의견은 책의 방향을 정하는 데 큰 도움이 됩니다!
Traceback (most recent call last):
  (... 생략 ...)
  File "c:\github\gpt_agent_2024_book\chapter14\v0401_vector_search_agent.py", line
  235, in communicator
    raise ValueError(f"Communicator가 아닌 agent가 대화를 시도하고 있습니다.\n {task_histo-
  ry[-1]}")
ValueError: Communicator가 아닌 agent가 대화를 시도하고 있습니다.
  agent='vector_search_agent' done=False description='HYBE와 JYP에 대한 정보를 수집하여 목
  차 작성에 필요한 자료를 준비합니다.' done_at=''

```

이 오류는 벡터 검색 에이전트가 생성된 작업에 대해 실제 작업을 수행하지 않고 완료 표시도 없이 바로 커뮤니케이터 에이전트에게 작업을 넘겨서 발생한 문제입니다. 커뮤니케이터 에이전트는 ‘현재 해야 할 작업은 벡터 검색 에이전트 담당인데 커뮤니케이터 에이전트에서 처리하고 있다.’는 오류 메시지를 출력한 것입니다.

이제 코드가 제대로 작동할 수 있도록 벡터 검색 에이전트의 RAG 기능을 구현해 봅시다.

벡터 검색 에이전트에 RAG 기능 구현하기

벡터 DB에서 검색하는 도구를 언어 모델이 자유롭게 활용할 수 있도록 `.bind_tools`를 이용해 RAG 기능을 구현하겠습니다. 이렇게 하면 언어 모델이 자신의 판단에 따라 여러 번 검색어를 바꾸며 벡터 검색을 할 수 있습니다.

1. 먼저 벡터 검색 에이전트가 이전 작업과 대화 내용을 바탕으로 목차 작성을 위한 정보를 벡터 검색을 통해 찾는 기능을 추가합니다.

벡터 검색 기능을 도구로 구현하기 (1)

book_writer.py

```
(... 생략 ...)
# 상태 정의
class State(TypedDict):
    messages: List[AnyMessage | str]
    task_history: List[Task]
    references: dict ①
(... 생략 ...)

def vector_search_agent(state: State):
    print("\n\n===== VECTOR SEARCH AGENT =====")

    tasks = state.get("task_history", [])
    task = tasks[-1]
    if task.agent != "vector_search_agent":
        raise ValueError(f"Vector Search Agent가 아닌 agent가 Vector Search Agent를
시도하고 있습니다.\n {task}")

    vector_search_system_prompt = PromptTemplate.from_template(
        """
너는 다른 AI Agent 들이 수행한 작업을 바탕으로,
목차(outline) 작성에 필요한 정보를 벡터 검색을 통해 찾아내는 Agent이다.

현재 목차(outline)를 작성하는 데 필요한 정보를 확보하기 위해,
다음 내용을 활용해 적절한 벡터 검색을 수행하라.

- 검색 목적: {mission}
-----
- 과거 검색 내용: {references}
-----
- 이전 대화 내용: {messages}
-----
- 목차(outline): {outline}
"""
    )

    mission = task.description
    references = state.get("references", {"queries": [], "docs": []})
    messages = state["messages"]
    outline = get_outline(current_path)
```

2

3

4

```

inputs = {
    "mission": mission,
    "references": references,
    "messages": messages,
    "outline": outline
}

llm_with_retriever = llm.bind_tools([retrieve])
vector_search_chain = vector_search_system_prompt | llm_with_retriever

search_plans = vector_search_chain.invoke(inputs)

for tool_call in search_plans.tool_calls:
    print('-----', tool_call)
(... 생략 ...)

```

- ❶ 벡터 검색 결과를 State에 보관하기 위해 State 클래스에 references 항목을 추가합니다. 자료형은 딕셔너리로 설정합니다.
- ❷ 가장 최근의 작업을 가져와 해당 작업을 vector_search_agent에서 처리할 수 있는지 확인합니다.
- ❸ 벡터 검색을 위한 시스템 프롬프트를 만듭니다. 이전에 검색한 내용이 있다면 이를 참고하는 입력값으로 references를 설정합니다. 또한 이전 대화 내용과 현재 목차도 적절한 쿼리를 생성하는 데 도움이 되므로 인풋에 추가합니다.
- ❹ 시스템 프롬프트에서 필요한 인풋값을 설정하는 코드입니다. 검색 목적인 mission은 supervisor에서 생성한 task의 description을 사용합니다. 검색 결과와 참고 자료를 담는 references는 state에 이미 존재하는 항목을 가져오고 항목이 없으면 queries와 docs 리스트를 키로 갖는 딕셔너리를 생성합니다. queries에는 벡터 검색을 할 때 사용한 쿼리들이, docs에는 검색된 문서(청크)가 담깁니다. 목차를 의미하는 outline은 이전에 만든 get_outline 함수를 이용해 추가합니다.
- ❺ tools.py 파일의 retrieve 함수는 @tool 데코레이터를 이용해 랭체인에서 바로 사용할 수 있도록 설정되어 있습니다. llm은 GPT-4o로 설정하고 .bind_tools() 를 이용해 retrieve 도구를 붙입니다. 그리고 vector_search_prompt와 연결해서 vector_search_chain을 만듭니다.
- ❻ 생성된 vector_search_chain에 인풋값을 넣어 실행한 후 결과를 확인합니다. 이 체인에서는 검색을 여러 번 수행할 수 있도록 여러 검색어를 만들어 실행할 계획을 세울 수 있습니다. 이 계획은 search_plans 변수에 담깁니다.
- ❼ search_plans에는 언어 모델이 검색하고 싶은 내용이 담깁니다. 여러 개를 검색할 수 있으므로 결과는 리스트 형태로 반환됩니다.

현재 버전에서는 여전히 오류가 발생하지만 `vector_search_agent`가 실행되는 동안에는 다음과 같은 메시지가 출력됩니다. 사용자가 'HYBE와 JYP 비교하는 책 쓰자. 우선 벡터 검색부터 해'라고 요청하면 `supervisor`는 HYBE와 JYP를 비교하는 책을 쓰는 데 필요한 정보를 검색하기 위해 `vector_search_agent`를 사용하는 작업을 생성합니다. 출력된 메시지에서 'VECTOR SEARCH AGENT'라고 표시된 부분은 `vector_search_agent`에서 검색해야 한다고 판단한 내용들입니다. 이 내용 중에서 `query` 문자열을 보면 HYBE와 JYP의 기업 규모, 경영 전략, 재무 상태 등 다양한 질문들이 목록으로 나열된 것을 확인할 수 있습니다.

User: HYBE와 JYP 비교하는 책 쓰자. 우선 벡터검색부터 해

===== SUPERVISOR =====

[Supervisor] agent='vector_search_agent' done=False description='HYBE와 JYP를 비교하는 책을 쓰기 위한 정보를 벡터 검색을 사용하여 수집한다.' done_at=''

===== VECTOR SEARCH AGENT =====

----- {'name': 'retrieve', 'args': {'query': 'HYBE company overview'}, 'id': 'call_me3IiG2le0ImhW0xddnIdjCl', 'type': 'tool_call'}

----- {'name': 'retrieve', 'args': {'query': 'JYP company overview'}, 'id': 'call_Q2NUclSgd8JKGBSOF4WdHl1', 'type': 'tool_call'}

----- {'name': 'retrieve', 'args': {'query': 'HYBE corporate strategy'}, 'id': 'call_L97gJfPdW87CP35cqoVTWoSl', 'type': 'tool_call'}

----- {'name': 'retrieve', 'args': {'query': 'JYP corporate strategy'}, 'id': 'call_0vt0dprUDYiIpKbkqMvqzGgR', 'type': 'tool_call'}

-

(... 생략 ...)

(이후 에러 발생 `ValueError: Communicator가 아닌 agent가 대화를 시도하고 있습니다.`)

2. 출력된 내용을 보면 `retrieve` 함수에 전달해야 하는 값들이 `args` 안에 들어 있습니다. 이 `args`를 그대로 `retrieve` 함수에 넣어서 결과를 반환합니다.

벡터 검색 기능을 tool로 구현하기 (2)

book_writer.py

```
def vector_search_agent(state: State):
    (... 생략 ...)

    search_plans = vector_search_chain.invoke(inputs)

    for tool_call in search_plans.tool_calls:
        print('-----', tool_call)
```



```

args = tool_call["args"]

query = args["query"]
retrieved_docs = retrieve(args)

references["queries"].append(query)
references["docs"] += retrieved_docs

unique_docs = []
unique_page_contents = set()

for doc in references["docs"]:
    if doc.page_content not in unique_page_contents:
        unique_docs.append(doc)
        unique_page_contents.add(doc.page_content)
references["docs"] = unique_docs

print('Queries:-----')
queries = references["queries"]
for query in queries:
    print(query)

print('References:-----')
for doc in references["docs"]:
    print(doc.page_content[:100])
    print('-----')

tasks[-1].done = True
tasks[-1].done_at = datetime.now().strftime('%Y-%m-%d %H:%M:%S')

new_task = Task(
    agent="communicator",
    done=False,
    description="AI팀의 진행상황을 사용자에게 보고하고, 사용자의 의견을
파악하기 위한 대화를 나눈다",
    done_at=""
)
tasks.append(new_task)

msg_str = f"[VECTOR SEARCH AGENT] 다음 질문에 대한 검색 완료: {queries}"
message = AIMessage(msg_str)
print(msg_str)

messages.append(message)

```

```

return {
    "messages": messages,
    "task_history": tasks,
    "references": references
}

```

- ❶ references에 검색 내용을 담아야 하므로 args의 값 중에서 query를 queries에 추가합니다. retrieved_docs로 받은 결과도 references에 docs로 추가합니다.
- ❷ 벡터 검색에 사용한 query가 다양하더라도 벡터 DB에 저장된 문서의 종류가 많지 않거나, 한 문서가 여러 질문에 대한 답을 모두 포함하고 있을 때, 같은 문서가 반복해서 나올 수 있습니다. 같은 문서가 다른 query에 대해 반복해서 나오더라도 중복없이 처리하기 위해 page_content를 set을 사용해 unique_page_contents에 담습니다. 이렇게 중복 없이 정리된 문서(체크)들만 references["docs"]에 담습니다.
- ❸ 검색한 질의들(queries)과 검색된 체크 문서들을 출력하는 코드입니다.
- ❹ 최근 생성된 Task를 done=True로 설정해서 완료된 상태로 처리합니다. 현재 vector_search_agent는 communicator와 연결되어 있습니다. communicator에서 새로운 작업을 받아서 처리해야 할 일이 생겼으므로 다음 할 일을 의미하는 Task 객체를 생성해 new_task 변수에 담습니다.
- ❺ 작업이 완료되면 vector_search_agent의 작업 후기를 AIMessage로 생성합니다. 이렇게 작성하면 다음 에이전트가 현재의 진행 상황을 더 잘 파악할 수 있습니다.
- ❻ vector_search_agent의 작업 결과를 상태에 업데이트하기 위해 반환합니다.

코드를 실행한 후 ‘우선 검색부터 해.’라고 메시지를 입력하여 vector_search_agent를 실행했습니다. HYBE와 JYP를 비교하기 위해 필요한 정보 5가지를 검색해서 그 결과를 출력했습니다. 5개의 질문에 대해 top_k를 5로 설정하여 25개의 체크가 검색되었고 중복 제거 후 References로 출력되었습니다.

앞에서 웹 검색 기능을 구현할 때 JYP와 HYBE에 관한 내용을 검색한 적이 없다면 벡터 DB에 두 회사와 관련된 내용이 저장되어 있지 않을 것입니다. 저는 경제와 영화에 관한 질문만 테스트해 본 상태이므로 다음과 같이 두 회사에 대한 내용은 검색되지 않았습니다. 이 부분은 추후에 웹 검색 에이전트를 추가하여 개선할 예정이므로 지금은 벡터 DB에서 검색을 할 수 있다는 것만 확인하면 됩니다.

User: HYBE와 JYP 비교하는 책 쓰자. 우선 검색부터 해

===== SUPERVISOR =====

[Supervisor] agent='vector_search_agent' done=False description='HYBE와 JYP를 비교하는 책의 목차 작성을 위한 정보를 벡터 DB에서 검색한다.' done_at=''

===== VECTOR SEARCH AGENT =====

----- {'name': 'retrieve', 'args': {'query': 'HYBE와 JYP의 회사 역사 비교', 'top_k': 5}, 'id': 'call_TKmdcaLQTX3600PCDSFBhFoA', 'type': 'tool_call'}
----- {'name': 'retrieve', 'args': {'query': 'HYBE와 JYP의 비즈니스 모델 분석', 'top_k': 5}, 'id': 'call_5SXAAFch50C3JEz7I6PtaPly', 'type': 'tool_call'}
----- {'name': 'retrieve', 'args': {'query': 'HYBE와 JYP의 아티스트 관리 방식 차이', 'top_k': 5}, 'id': 'call_yXcaUqlh3jRDU4E88pbi9EVu', 'type': 'tool_call'}
----- {'name': 'retrieve', 'args': {'query': 'HYBE와 JYP의 글로벌 진출 전략', 'top_k': 5}, 'id': 'call_PWwzX2jFW8oEwpjXyj7fmmDb', 'type': 'tool_call'}
----- {'name': 'retrieve', 'args': {'query': 'HYBE와 JYP의 음악 스타일 및 프로덕션 비교', 'top_k': 5}, 'id': 'call_XbjWsWXjURdYwhbsyxsn9tLJ', 'type': 'tool_call'}

Queries:-----

HYBE와 JYP의 회사 역사 비교

HYBE와 JYP의 비즈니스 모델 분석

HYBE와 JYP의 아티스트 관리 방식 차이

HYBE와 JYP의 글로벌 진출 전략

HYBE와 JYP의 음악 스타일 및 프로덕션 비교

References:-----

References:-----

- 현재 수출 경기 호조의 핵심인 반도체 산업의 단기 사이클(시장의 수급)과 중 장기 사이클(주력 품목의 변화)의 움직임을 예의주시하고 우리 기업들의 대응 여력을 확충하는 데에 주... 생략 ...

주: OECD의 선진국은 OECD 회원국, 신흥개도국은 비OECD 회원국임. 2-2. 국내 경기 : 확대되고 있는 성장경로 이탈 가능성 ○(전반적인 경기 활력 저하) 내수 부진이

(건설투자) 금융조달 비용의 하락, 부동산 PF 부실 문제의 점진적인 완화, 수도권 중심 부동산 경기 회복세 지속, 정부의 주택공급 확대책 현실화 등으로 하반기로 갈수록 경기 여건

(고용) 전반적으로 실업률이 소폭 상승한 가운데 연령별로는 청년층, 산업별로는 제 조업과 건설업의 고용 창출력 위축 현상이 뚜렷하게 나타나고 있다. 10월 실업률은 2.3%로 전년

전망 - 설비투자 : 회복세 강화. 연간 2.7%(상반기 5.3%, 하반기 0.1%) 「금리 하락에 따르는 투자심리 개선, AI 관련 투자 확대 등으로 회복세가 다소 강화될 것으로

===== COMMUNICATOR =====

AI: 현재 AI 팀은 HYBE와 JYP를 비교하는 책의 목차 작성을 위해 필요한 정보를 수집하고 있습니다. 벡터 검색 에이전트를 통해 다음과 같은 주제에 대한 정보를 확보했습니다:

1. HYBE와 JYP의 회사 역사 비교
2. HYBE와 JYP의 비즈니스 모델 분석
3. HYBE와 JYP의 아티스트 관리 방식 차이
4. HYBE와 JYP의 글로벌 진출 전략
5. HYBE와 JYP의 음악 스타일 및 프로덕션 비교

이 주제들을 중심으로 책의 목차를 구성할 계획입니다. 사용자의 의견이나 추가로 다루고 싶은 주제가 있다면 말씀해 주세요. 이를 반영하여 더욱 풍부한 내용을 담을 수 있도록 하겠습니다.

----- MESSAGE COUNT 5

User:

벡터 DB 검색도 잘 이루어졌고 중복도 잘 처리된 것 같습니다. 현재 문제점은 벡터 DB 내에 관련 문서가 거의 없다는 점입니다. 검색 결과가 제한될 수밖에 없겠죠. 벡터 DB에 더 많은 문서를 추가하면 더 풍부한 검색 결과를 얻을 수 있을 것입니다. 이와 관련된 내용은 14-4절에서 자세히 알아보겠습니다.

참고 자료를 state.json에 저장하기

마지막으로 벡터 검색 결과로 가져온 참고 자료를 state.json 파일에 저장하여 현재 상태를 확인할 수 있도록 하겠습니다.

references를 저장하는 코드를 작성합니다. 특히 state의 references 항목에 담긴 문서 청크는 랭체인이 Document 객체로 되어 있어서 이를 바로 딕셔너리로 만들 수 없습니다. 그래서 문서의 metadata를 추출하여 저장합니다.

참고 자료도 state.json에 저장하기

utils.py

```
def save_state(current_path, state):
    if not os.path.exists(f"{current_path}/data"):
        os.makedirs(f"{current_path}/data")

    state_dict = {}

    messages = [(m.__class__.__name__, m.content) for m in state["messages"]]

    state_dict["messages"] = messages
    state_dict["task_history"] = [task.to_dict() for task in state.get("task_history",
    [])]
```



```
# references
references = state.get("references", {"queries": [], "docs": []})
state_dict["references"] = {
    "queries": references["queries"],
    "docs": [doc.metadata for doc in references["docs"]]
}

with open(f"{current_path}/data/state.json", "w", encoding='utf-8') as f:
    json.dump(state_dict, f, indent=4, ensure_ascii=False)
```

이 코드를 실행하면 다음과 같이 state.json 파일에 기록이 남아 있습니다.

```
{
  "messages": [
    [
      "SystemMessage",
      "\n          너희 AI들은 사용자의 요구에 맞는 책을 쓰는 작가팀이다.\n
사용자가 사용하는 언어로 대화하라.\n\n          현재시각은 2025-03-06 00:40:23이다.\n\n
"
    ],
    [
      "HumanMessage",
      "HYBE와 JYP 비교하는 책을 써줘. 우선 검색부터 하자"
    ],
    [
      "AIMessage",
      "[Supervisor] agent='vector_search_agent' done=False description='사용자가
요청한 HYBE와 JYP 비교 책의 목차 작성을 위한 정보 확보가 필요하므로, 벡터 DB를 검색하여 필요한 정
보를 수집한다.' done_at=''"
    ],
    (... 생략 ...)
  ],
  "references": {
    "queries": [
      "HYBE company overview",
      "JYP company overview",
      "Comparison between HYBE and JYP in business strategies",
      "HYBE and JYP financial performance overview",
      "Artists and music style differences between HYBE and JYP"
    ],
    "docs": [
      {
        "source": "https://www.hri.co.kr/upload/board/2887010266_WZ-
R5c0kj_20241210123438.pdf",
```

```
        "title": "[PDF] 2025년 한국경제 수정전망"
    },
    {
        "source": "https://eiec.kdi.re.kr/policy/domesticView.do?ac=0000191361",
        "title": "2025년 거시경제 전망 | 국내연구자료 | KDI 경제교육·정보센터"
    },
    (... 생략 ...)
```

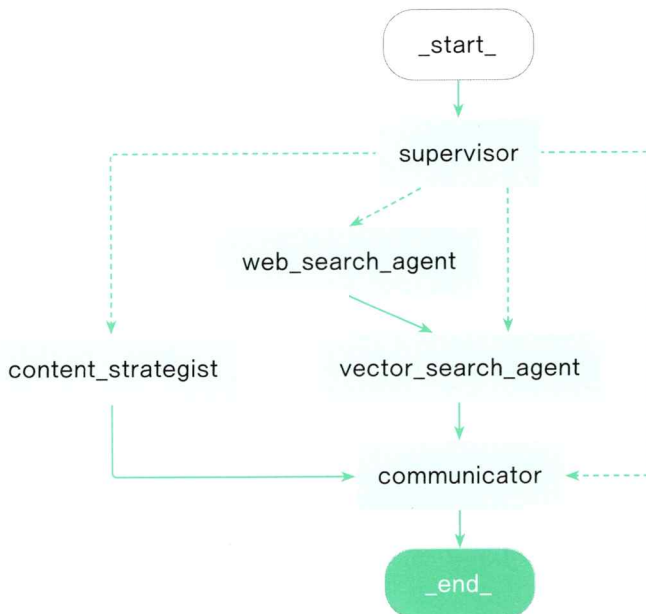
14-4 부족한 정보를 검색하는 웹 검색 에이전트

벡터 검색 에이전트를 보완하는 웹 검색 에이전트를 만들고 목차 작성에 웹 검색 결과를 활용해 보겠습니다.

부족한 정보를 찾아 주는 웹 검색 에이전트

14-3절에서 벡터 검색을 통해 벡터 DB의 문서를 찾는 에이전트는 만들었지만 벡터 DB에 저장된 내용이 적어서 실용성이 떨어졌습니다. 벡터 DB에 정보가 부족하다면 인터넷에 검색해 벡터 DB에 추가해야 합니다. 예전에는 사람이 직접 검색해야 했지만 이제는 AI 에이전트를 활용할 수 있습니다. 웹 검색을 통해 벡터 DB에 내용을 채워 주는 웹 검색 에이전트 `web_search_agent`를 만들어 봅시다.

필요한 함수는 14-3절에서 대부분 만들어 두었으므로 이번 절에서는 웹 검색 후 웹 페이지 문서를 벡터 DB에 자동으로 저장하고 벡터 검색 에이전트 `vector_search_agent`에서 벡터 검색을 하는 경로를 추가하는 작업해 보겠습니다.



벡터 검색 전에 `web_search_agent`로 벡터 DB 자료를 확보하는 그래프의 구조

Do it! 실습 웹 검색 에이전트 web_search_agent 만들기

■ 결과 파일: sec04/book_writer_0.py, models.py

웹 검색 에이전트 web_search_agent을 만들어 보겠습니다. book_writer.py 파일에 이어서 작성하세요.

1. web_search_agent의 구조는 14-3절에서 만든 vector_search_agent와 거의 동일합니다. 앞서 tools.py 파일에 작성한 web_search에는 @tool 데코레이터를 사용하여 랭체인과 랭그래프에서 바로 활용할 수 있도록 설정해두었습니다. 이를 .bind_tools()를 이용해 언어 모델에 연결하여 필요에 따라 웹 검색을 여러 번 할 수 있게 합니다. 그 결과는 add_web_pages_json_to_chroma 함수를 통해 벡터 DB에 저장합니다.

웹 검색 에이전트 코드 만들기

book_writer.py

```
(... 생략 ...)  
  
from utils import save_state, get_outline, save_outline  
from models import Task  
from tools import retrieve, web_search, add_web_pages_json_to_chroma —10  
  
from datetime import datetime  
(... 생략 ...)  
  
# 목차를 작성하는 노드(agent)  
def content_strategist(state: State):  
    print("\n\n===== CONTENT STRATEGIST =====")  
    (... 생략 ...)  
  
    return {  
        "messages": messages,  
        "task_history": task_history  
    }  
  
def web_search_agent(state: State): —1  
    print("\n\n===== WEB SEARCH AGENT =====")  
  
    tasks = state.get("task_history", [])  
    task = tasks[-1]  
  
    if task.agent != "web_search_agent":  
        raise ValueError(f"Web Search Agent가 아닌 agent가 Web Search Agent를 시도하고  
있습니다.\n {task}")
```

2


```

web_search_system_prompt = PromptTemplate.from_template(
    """
    너는 다른 AI Agent 들이 수행한 작업을 바탕으로,
    목차(outline) 작성에 필요한 정보를 웹 검색을 통해 찾아내는 Web Search Agent이다.

    현재 부족한 정보를 검색하고, 복합적인 질문은 나눠서 검색하라.

    - 검색 목적: {mission}
    -----
    - 과거 검색 내용: {references}
    -----
    - 이전 대화 내용: {messages}
    -----
    - 목차(outline): {outline}
    -----
    - 현재 시각 : {current_time}
    """
)

```

3

```

messages = state.get("messages", [])

```

4

```

inputs = {
    "mission": task.description,
    "references": state.get("references", {"queries": [], "docs": []}),
    "messages": messages,
    "outline": get_outline(current_path),
    "current_time": datetime.now().strftime('%Y-%m-%d %H:%M:%S')
}

```

5

```

llm_with_web_search = llm.bind_tools([web_search])

```

6

```

web_search_chain = web_search_system_prompt | llm_with_web_search

```

7

```

search_plans = web_search_chain.invoke(inputs)

```

8

```

queries = []

```

9

```

for tool_call in search_plans.tool_calls:
    print('----- web search -----', tool_call)
    args = tool_call["args"]

    queries.append(args["query"])

```

10

```

# 검색 결과 JSON 파일 경로 가져오기
_, json_path = web_search.invoke(args)
print('json_path:', json_path)

# JSON 파일을 크로마DB에 추가
add_web_pages_json_to_chroma(json_path)

tasks[-1].done = True
tasks[-1].done_at = datetime.now().strftime('%Y-%m-%d %H:%M:%S')

task_desc = "AI팀이 쓸 책의 세부 목차를 결정하기 위한 정보를 벡터 검색을 통해 찾아낸다."
task_desc += f" 다음 항목이 새로 추가되었다\n: {queries}"

new_task = Task(
    agent="vector_search_agent",
    done=False,
    description=task_desc,
    done_at=""
)

tasks.append(new_task)

msg_str = f"[WEB SEARCH AGENT] 다음 질문에 대한 검색 완료: {queries}"
messages.append(AIMessage(msg_str))

return {
    "messages": messages,
    "task_history": tasks
}
(... 생략 ...)
```

- ❶ web_search_agent 함수를 만듭니다. 이는 웹 검색을 하는 새로운 에이전트 노드입니다.
- ❷ 작업 목록에서 최근 작업(task)을 확인하고 web_search_agent가 처리할 작업이 맞는지 확인합니다.
- ❸ 시스템 프롬프트를 정의합니다. web_search_agent의 주요 임무는 목차를 생성하는 데 필요하지만 벡터 DB에는 부족한 정보를 찾아서 보강하는 것입니다. 따라서 목차(outline)뿐만 아니라 state에서 관리하는 references도 입력값으로 받습니다. 현재 시각(current_time)을 추가한 이유는 언어 모델이 최신 자료를 검색할 때 검색어에 해당 언어 모델이 생성된 연도를 입력하는 문제가 있었기 때문입니다. 이렇게 하면 현재 시각을 활용해 최신 정보를 제대로 검색할 수 있습니다. 검색 목적(mission)은 supervisor에서 Task를 생성할 때 description에 채워 썼던 내용을 사용합니다. 그리고 프롬프트에 여러 가지 키워드를 합치지 않고 개별 키워드로 나눠서 검색하도록 프롬프트를 작성합니다. 예를 들어 'JYP와 HYBE의 연혁' 보다 'JYP의 연혁', 'HYBE의 연혁' 처럼 따로 검색해야 자료가 많을 테니까요.
- ❹ 이전 대화를 가져와 messages에 담습니다. 이 messages는 inputs에 활용하기도 하지만 나중에 web_search_agent의 작업 후기도 추가할 예정이므로 따로 저장해 두었습니다.

- 5 인풋 자료를 준비합니다. 3에서 정의한 시스템 프롬프트에 들어가야 할 내용들을 딕셔너리 형태로 만듭니다.
- 6 언어 모델 llm과 전에 만들어 둔 web_search 도구를 .bind_tools로 연결합니다.
- 7 시스템 프롬프트와 결합하여 web_search_chain을 만듭니다.
- 8 web_search_chain.invoke로 언어 모델이 판단해 검색할 리스트를 받아 옵니다.
- 9 web_search_agent 에서 어떤 작업을 했는지 메시지로 남기기 위해 queries를 빈 리스트로 만들어 놓고 검색 결과를 계속 추가합니다.
- 10 GPT가 원하는 검색어가 args에 담겨 있으므로 앞서 만들었던 web_search로 검색 후 그 결과가 담긴 JSON 파일을 add_web_pages_json_to_chroma 함수를 이용해 크로마 DB에 저장합니다.
- 11 가장 최근의 작업을 done=True로 처리하고 새로운 Task를 추가합니다. 새로 만들 Task는 vector_search_agent로 지정하고 필요한 메시지를 description에 넣습니다. 이때 description에 들어갈 메시지에 앞서 모아두었던 queries의 값을 넣어 줍니다. 그래야 그 항목에 대해 벡터 검색을 집중적으로 할 수 있으니까요.
- 12 messages에도 web_search_agent 작업 후기에 해당하는 메시지를 추가합니다.
- 13 마지막으로 상태를 업데이트하기 위해 messages와 task_history를 반환합니다.

2. supervisor가 web_search_agent도 선택할 수 있도록 프롬프트 메시지에 web_search_agent에 대한 설명을 추가합니다. 이 부분만 수정하는 것으로는 충분하지 않으며 models.py 파일의 Task에 새로운 항목을 추가해야 합니다. 이는 뒤에서 구현하겠습니다.

웹 검색 에이전트를 supervisor에 등록하기

book_writer.py

(... 생략 ...)

```
def supervisor(state: State):
    print("\n\n===== SUPERVISOR =====")

    # 시스템 프롬프트 정의
    supervisor_system_prompt = PromptTemplate.from_template(
        """
        너는 AI 팀의 supervisor로서 AI 팀의 작업을 관리하고 지도한다.
        사용자가 원하는 책을 써야 한다는 최종 목표를 염두에 두고,
        사용자의 요구를 달성하기 위해 현재 해야 할 일이 무엇인지 결정한다.

        supervisor가 활용할 수 있는 agent는 다음과 같다.
        - content_strategist: 사용자의 요구 사항이 명확해졌을 때 사용한다. AI 팀의 콘텐츠 전략
        을 결정하고, 전체 책의 목차(outline)를 작성한다.
        - communicator: AI 팀에서 해야 할 일을 스스로 판단할 수 없을 때 사용한다. 사용자에게 진
        행 상황을 보고하고, 다음 지시를 물어본다.
        - web_search_agent: 웹 검색을 통해 목차(outline) 작성에 필요한 정보를 확보한다.
        - vector_search_agent: 벡터 DB 검색을 통해 목차(outline) 작성에 필요한 정보를 확보한다.

        아래 내용을 고려하여, 현재 해야할 일이 무엇인지, 사용할 수 있는 agent가 무엇인지 단답으로
        말하라.
```

```

-----
previous_outline: {outline}
-----

messages:
{messages}
"""

)

(... 생략 ...)

```

3. 새로 만든 `web_search_agent`를 그래프에 추가합니다. 노드를 추가하고 `supervisor`의 프롬프트 메시지에 `web_search_agent`를 추가했으므로 `conditional_edge`에도 이를 추가합니다. 또한 `web_search_agent` 이후에는 바로 `vector_search_agent`로 이어지도록 연결합니다.

그래프 수정하기

book_writer.py

```

(... 생략 ...)

# 상태 그래프 정의
graph_builder = StateGraph(State)

# 노드
graph_builder.add_node("supervisor", supervisor)
graph_builder.add_node("communicator", communicator)
graph_builder.add_node("content_strategist", content_strategist)
graph_builder.add_node("vector_search_agent", vector_search_agent)
graph_builder.add_node("web_search_agent", web_search_agent)

# 엣지
graph_builder.add_edge(START, "supervisor")
graph_builder.add_conditional_edges(
    "supervisor",
    supervisor_router,
    {
        "content_strategist": "content_strategist",
        "communicator": "communicator",
        "vector_search_agent": "vector_search_agent",
        "web_search_agent": "web_search_agent"
    }
)

```



```
)

graph_builder.add_edge("content_strategist", "communicator")
graph_builder.add_edge("web_search_agent", "vector_search_agent")
graph_builder.add_edge("vector_search_agent", "communicator")
graph_builder.add_edge("communicator", END)

graph = graph_builder.compile()

(... 생략 ...)
```

4. 마지막으로 models.py 파일에 Task 모델의 새로운 노드인 web_search_agent가 선택지로 추가될 수 있도록 agent에 추가합니다. description에도 web_search_agent를 언제 활용하면 좋은지 설명을 추가하여, 해당 AI 에이전트의 사용 목적과 상황을 명확하게 정의했습니다.

Task 모델에 새 에이전트 web_search_agent 선택지 추가

models.py

```
class Task(BaseModel):
    agent: Literal[
        "content_strategist",
        "communicator",
        "web_search_agent",
        "vector_search_agent",
    ] = Field(
        ...,
        description="""
        작업을 수행하는 agent의 종류.
        - content_strategist: 콘텐츠 전략을 수립하는 작업을 수행한다. 사용자의 요구 사항이 명확
        해졌을 때 사용한다. AI 팀의 콘텐츠 전략을 결정하고, 전체 책의 목차(outline)를 작성한다.
        - communicator: AI 팀에서 해야 할 일을 스스로 판단할 수 없을 때 사용한다. 사용자에게 진
        행 상황을 보고하고, 다음 지시를 물어본다.
        - web_search_agent: 웹 검색을 통해 목차(outline) 작성에 필요한 정보를 확보한다.
        - vector_search_agent: 벡터 DB 검색을 통해 목차(outline) 작성에 필요한 정보를 확보한다.
        """
    )

(... 생략 ...)
```

현재 상태에서 실행하면 AI 에이전트가 자신이 필요하다고 판단하는 정보들을 웹에서 검색한 후 벡터 DB에서 검색해서 활용합니다. 현재는 웹 검색을 먼저 진행하지만 벡터 DB가 충분히 커지면 벡터 DB에서 우선 검색하고 필요한 정보가 없을 때 웹 검색을 하도록 설정할 수도 있겠죠.

User: HYBE와 JYP 비교하는 책 쓰자. 우선 검색부터 해

===== SUPERVISOR =====

[Supervisor] agent='web_search_agent' done=False description='HYBE와 JYP 비교에 필요한 정보를 웹 검색을 통해 확보한다.' done_at=''

===== WEB SEARCH AGENT =====

----- web search ----- {'name': 'web_search', 'args': {'query': 'HYBE 회사 정보 2024'}, 'id': 'call_zzWTYfRF1RbKnkOwVmnsSnaF', 'type': 'tool_call'}

json_path: c:\github\gpt_agent_2024_book\chapter14\data/resources_2024_1210_220038.json

Documents를 Chroma DB에 저장합니다.

(... 생략 ...)

'https://themusicnetwork.com/hybe-corporation-kpop-2024/'}

Splitting documents...

5개의 문서를 1000자 크기로 중첩 100자로 분할합니다.

총 52개의 문서로 분할되었습니다.

----- web search ----- {'name': 'web_search', 'args': {'query': 'JYP 회사 정보 2024'}, 'id': 'call_c2lm7iAOTl9bhOMEQVap20Zi', 'type': 'tool_call'}

json_path: c:\github\gpt_agent_2024_book\chapter14\data/resources_2024_1210_220048.json

Documents를 Chroma DB에 저장합니다.

{'title': 'JYP Ent., 2024년 3분기 실적 발표: 견고한 성장세 지속', 'source': 'https://dartgpt.ai/filings/20241113000443'}

(... 생략 ...)

총 11개의 문서로 분할되었습니다.

===== VECTOR SEARCH AGENT =====

----- {'name': 'retrieve', 'args': {'query': 'HYBE 회사 정보 2024'}, 'id': 'call_W77EJUdTqxTi89MvzgznqNL', 'type': 'tool_call'}

(... 생략 ...)

----- {'name': 'retrieve', 'args': {'query': 'HYBE와 JYP의 주식 시장 성과 2024'}, 'id': 'call_YpJL640dHfT02MfFwZkl5R1d', 'type': 'tool_call'}

Queries:-----

HYBE 회사 정보 2024

JYP 회사 정보 2024

HYBE 엔터테인먼트 아티스트 목록 2024

JYP 엔터테인먼트 아티스트 목록 2024

HYBE와 JYP의 수익 비교 2024

HYBE와 JYP의 주식 시장 성과 2024

References:-----

하이브영문 명칭HYBE Co., Ltd. 국가 대한민국 행정구 속령설립일2005년 2월 4일 (19주년)[1]업종명음악
및 기타 오디오물 출판업창업주방시혁총수최대 주주 임원진이사회

(... 생략 ...)

[VECTOR SEARCH AGENT] 다음 질문에 대한 검색 완료: ['HYBE 회사 정보 2024', 'JYP 회사 정보 2024', 'HYBE 엔터테인먼트 아티스트 목록 2024', 'JYP 엔터테인먼트 아티스트 목록 2024', 'HYBE와 JYP의 수익 비교 2024', 'HYBE와 JYP의 주식 시장 성과 2024']

===== COMMUNICATOR =====

AI: 안녕하세요! 현재 AI팀은 HYBE와 JYP를 비교하는 책을 집필 중에 있습니다. 최근 진행 상황으로는 HYBE와 JYP의 2024년 회사 정보, 엔터테인먼트 아티스트 목록, 수익 비교, 주식 시장 성과 등을 조사했습니다. 이제 사용자의 의견을 바탕으로 내용을 구체화해 나가고자 합니다.

혹시 이 책에 포함되었으면 하는 구체적인 주제나 추가로 알고 싶으신 부분이 있으신가요? 또는 다른 의견이 있으시면 말씀해 주세요!

Do it! 실습 목차에 검색 결과 활용하기

■ 결과 파일: sec04/book_writer.py

벡터 DB에서 필요한 자료를 검색하고 부족한 자료는 웹에서 검색하는 기능은 갖추었지만 아직 목차에는 이 정보가 반영되지 않습니다. 그 이유는 목차를 작성하는 `content_strategist`의 프롬프트에 관련 문서가 제공되지 않기 때문입니다. 이 문제를 해결하기 위해 `content_strategist`를 수정해 보겠습니다.

프롬프트에 참고 자료를 활용하라는 문구를 추가하고 인풋 자료에도 `references`로 포함시켜 관련 자료들이 목차 작성에 반영될 수 있도록 합니다.

```
def content_strategist(state: State):
    print("\n\n===== CONTENT STRATEGIST =====")

    # 시스템 프롬프트 정의
    content_strategist_system_prompt = PromptTemplate.from_template(
        """
        너는 책을 쓰는 AI팀의 콘텐츠 전략가(Content Strategist)로서,
        이전 대화 내용을 바탕으로 사용자의 요구사항을 분석하고, AI팀이 쓸 책의 세부 목차를 결정한다.

        지난 목차가 있다면 그 버전을 사용자의 요구에 맞게 수정하고, 없다면 새로운 목차를 제안한다.
        목차를 작성하는 데 필요한 정보는 "참고 자료"에 있으므로 활용한다.

        -----
        - 지난 목차: {outline}
        -----
        - 이전 대화 내용: {messages}
        -----
        - 참고 자료: {references}
        """
    )

    # 시스템 프롬프트와 모델 연결
    content_strategist_chain = content_strategist_system_prompt | llm | StrOutputParser()

    messages = state["messages"] # 상태에서 메시지 가져오기
    outline = get_outline(current_path) # 저장된 목차 가져오기

    # 입력값 정의
    inputs = {
        "messages": messages,
        "outline": outline,
        "references": state.get("references", {"queries": [], "docs": []})
    }
```

이 코드를 실행해 보면 참고 자료를 활용해 목차를 작성할 때도 있고 그렇지 않은 경우도 있습니다. 벡터 검색과 목차 작성을 상황에 따라 자동으로 처리하면 좋겠지만 현재 상태에서는 사용자가 일일이 지시를 내려야만 합니다. 이를 해결하려면 근본적인 변화가 필요할 것 같습니다. 사람이 상황을 판단하고 다음 작업을 지시하는 과정을 인공지능에게 넘길 수 있으면 좋겠다는 생각이 들지 않나요? 이 내용은 다음 장에서 다루겠습니다.